TRANSFORMER HYPERPARAMETERS

From guesswork to evidence

A practical course in architecture, optimization and experimental judgment

CAPACITY

OPTIMIZATION

EVIDENCE

Prepared for Manohar | 31 October 2026

150 pages / 28 chapters / worked examples and exercises

How to use this guide

This guide is written for a student who has built a Transformer and wants to replace arbitrary choices with a repeatable method. It assumes basic Python, matrices, derivatives and the idea of gradient descent. The focus is dense Transformers: decoder-only language models and conventional encoder-decoder models, with an introduction to fine-tuning and deployment.

Read the first four chapters before treating any suggested number as a recommendation. Then study architecture, optimization and evaluation in that order. Each chapter has five short lessons. Work through the calculations and decision exercises before reading their answers.

Every proposed configuration is illustrative unless explicitly described as a mathematical identity under stated assumptions. No model was trained to generate the hypothetical experiment results in this guide. The small-model and near-80M plans are standalone examples, not an audit of your SoC repository or a hardware benchmark.

The web lookup service was unavailable during preparation. The primary-source reading list points to established papers and official documentation, but links and current software APIs were not live-verified. Verify version-specific behavior in your installed environment. The arithmetic and standalone scheduler examples were checked locally. PyTorch integration sketches were syntax-checked but not executed: PyTorch was unavailable.

GOAL: You should finish able to justify a configuration, design a comparison, diagnose failure and revise a decision. No fixed handbook can remove every future uncertainty; it can teach you how to resolve it.

Contents 1 / 2

01 The decision process 7

02 Data, targets and tokenization 12

03 Transformer anatomy and counting 17

04 Compute, memory and token budgets 22

05 Choosing model width 27

06 Choosing the number of blocks 32

07 Choosing attention heads 37

08 Feed-forward width and activation 42

09 Context length and positional information 47

10 Encoder-decoder architecture choices 52

11 Initialization, normalization and residuals 57

12 Gradients and optimizer intuition 62

13 Selecting the learning rate 67

14 Warmup and learning-rate schedules 72

Notation: page 5 | Decision protocol: page 6

Contents 2 / 2

15 When and how to switch learning rate 77

16 Batch size and gradient accumulation 82

17 AdamW, decay and optimizer details 87

18 Regularization and training duration 92

19 Precision, memory and throughput 97

20 Evaluation and uncertainty 102

21 Diagnosing training curves 107

22 Designing an efficient hyperparameter search 112

23 A small decoder-only project 117

24 Designing a model near 80M parameters 122

25 Fine-tuning and parameter-efficient adaptation 127

26 Inference settings and deployment constraints 132

27 Implementation patterns and checks 137

28 Capstone: become an evidence-driven practitioner 142

Glossary: 147 | Reading list: 148-149 | Desk reference: 150

Notation and assumptions

L: number of repeated Transformer blocks. L_e and L_d: encoder and decoder block counts. d: residual-stream width. h or h_q: query-head count. h_kv: key/value-head count. d_h: dimension per head. Standard equal-head MHA commonly uses d = h d_h.

f: feed-forward intermediate width. V: vocabulary size. T: sequence or context length. B: sequence batch size for the scope stated. N: parameter count under the stated accounting convention. D: processed training-token exposures, unless explicitly called unique tokens.

eta: learning rate. lambda: weight-decay coefficient. beta1 and beta2: Adam moment coefficients. S: planned optimizer updates. W: warmup updates. tau: sampling temperature. A microbatch is one forward/backward batch; an effective batch combines the gradients for one optimizer update.

NLL: negative log-likelihood. Natural logarithms are used for cross-entropy and perplexity unless stated otherwise. M means one million and B in a numeric quantity such as 1.6B tokens means one billion. MiB and GiB are binary storage units; MB and GB are decimal.

FORMULA SCOPE: The approximation Vd + 12Ld^2 assumes a tied-embedding decoder, ordinary dense MHA and a plain two-matrix feed-forward network with f = 4d. It excludes biases, normalizations, learned positions and other implementation details. It is not a formula for every Transformer.

All numerical experiment outcomes described as hypothetical or illustrative are teaching examples. Plots in the guide visualize equations and stated assumptions; they are not measured model benchmarks.

The eight-step protocol

1. Define the task. Choose the primary validation metric, deployment requirements and maximum training budget. Decide whether comparisons should be equal-token, equal-time or both.

2. Audit the data. Inspect targets, shifts, masks, tokenization, splits, duplicates and valid-token counts. Keep the final test separate from configuration selection.

3. Validate a small baseline. Check finite values and actual parameter updates. Overfit a tiny batch. Verify evaluation and checkpoint recovery.

4. Measure feasibility. Count actual parameters and benchmark a complete training step. Record peak memory, valid tokens per second and evaluation overhead.

5. Tune optimization. Bracket peak learning rate, define warmup and decay, and choose effective batch deliberately. Examine training and validation together.

6. Test capacity and context. Compare a few widths, depths, head partitions and lengths with clear controls. Retune optimization modestly after major changes.

7. Diagnose and refine. Use the curves to choose between more training, better data, regularization, a schedule intervention or a different architecture. Preserve control checkpoints.

8. Confirm and report. Repeat close finalists where useful, freeze checkpoint selection and decoding, evaluate the test, and report cost and uncertainty alongside quality.

RULE: Change a number because it tests a hypothesis. If a result cannot change your next decision, ask whether the experiment is worth its cost.

Stop asking for the magic number

A hyperparameter is a choice made outside ordinary gradient updates. The weights are learned; width, depth, batch size, learning rate and the training horizon are selected. Some choices define which functions the model can represent. Others determine whether training can find useful weights within your budget. Treating all of them as interchangeable numbers hides the real problem.

Your goal is not to memorize one ideal configuration. It is to make a defensible choice, predict its consequences, measure them and revise the choice. A professional can explain why a run was worth attempting and what was learned when it failed. Uncertainty remains even in excellent experiments.

Separate three questions: Does the implementation work? Can this model learn the training task? Does the resulting model generalize and meet the deployment constraint? A larger model will not repair shifted labels. A lower learning rate will not remove train-test leakage. More epochs will not guarantee better validation performance.

DECISION: Before changing a number, write the failure you are trying to explain and one observation that would contradict your explanation.

A hierarchy of choices

Start with the task and evaluation protocol. Then establish data quality, tokenization and the amount of useful training data. Next choose a model family and an affordable architecture. Only after correctness checks should you spend heavily on optimizer and schedule comparisons. This ordering is a practical heuristic, not a theorem: discovered bugs can send you back to an earlier step.

Some variables deserve early attention because they affect nearly every experiment. Peak learning rate, batch size, token budget and model size often have broad effects. Exact head count, a tiny dropout change or the fourth decimal of weight decay are usually poor first priorities for an unvalidated project.

Freeze a baseline configuration in a file. Record inherited defaults explicitly. A value left out of your configuration is still a choice made somewhere in the software. Changing library versions can silently change the effective experiment.

EXAMPLE: If validation loss is flat, first verify labels, masks, learning rate actually applied and parameter updates. Searching 4, 8 and 16 heads before those checks produces expensive ambiguity.

Constraints are part of the objective

Suppose you can afford ten GPU-hours. You could spend all of them on one large model, or reserve time for small pilots, failed runs and a final confirmation. The second plan often produces more reliable knowledge even if its final model is smaller. Search itself consumes compute and must be budgeted.

Write an optimization statement: minimize validation negative log-likelihood subject to peak memory below the available limit, training time below the budget and inference latency below the product requirement. If translation quality is the real goal, include a translation metric and human inspection instead of relying solely on loss.

FORMULA: choose configuration c to minimize estimated validation risk R(c), subject to memory(c) <= M and total experiment cost <= C.

The estimated risk contains sampling noise and training randomness. The lowest observed value is not automatically the best true value. Keep enough budget to repeat close finalists and inspect whether a tiny win matters in practice.

CHECK: Can you state your primary metric, constraints and minimum useful improvement in one paragraph? If not, architecture tuning is premature.

A decision log that actually helps

Use one row per proposed experiment: hypothesis, controlled variables, changed variables, budget, predicted outcome, stopping rule, observed result and next action. Add code revision, data version, random seed and the full resolved configuration. This is more useful than a folder of screenshots named final2.

A good hypothesis is directional and falsifiable. For example: the model is optimization-limited, so reducing peak learning rate while keeping the token budget fixed should remove spikes and improve validation loss. A weak hypothesis is: maybe eight heads are better. Improve it by specifying what head dimension changes and how you will compare fairly.

Distinguish observation from interpretation. Observation: the global gradient norm rises sharply before loss becomes non-finite. Interpretation: the learning rate may be too high. Alternative explanations include a malformed batch or precision overflow. Log both so that a plausible story does not become an assumed fact.

PRACTICE: Write a decision log for a run that trains smoothly but produces repetitive text. Include at least two explanations and the cheapest test separating them.

Worked decision: a disappointing run

You train a six-block decoder. Training loss decreases from 6.0 to 3.1, while validation loss decreases from 6.1 to 3.4 and then rises to 3.6. A sample looks fluent but repeats phrases. You propose doubling the blocks and increasing temperature.

The validation rise suggests overfitting or distribution differences, but first compare losses under equivalent evaluation conditions. Training-mode dropout can distort the apparent gap. Evaluate a fixed training subset and the validation set in evaluation mode, with the same token weighting. Check whether the validation split duplicates training documents.

If the gap persists, preserve the best validation checkpoint. Test additional clean data, reduced training duration or appropriate regularization before adding capacity. Temperature can reduce visible repetition but does not establish that the model learned a better distribution. It is a decoding choice, not a repair for training quality.

ANSWER: The next experiment should isolate the generalization problem. The evidence does not justify more depth yet. Your final log should say what evidence would make a larger model reasonable: persistent high training loss after optimization has been checked, for example.

The model can only optimize the supplied task

For a causal language model, the input prefix predicts the next token. If the sequence is [A, B, C, D], a common training pair is input [A, B, C] and targets [B, C, D]. The causal mask prevents information from future input positions reaching earlier predictions. A model trained to copy the current input can achieve impressive loss while learning the wrong task.

For translation, the encoder receives the source sentence, and the decoder receives a shifted target sequence beginning with a start token. Targets include the next target token and normally an end token. Padding should not contribute to the objective. The decoder's self-attention must be causal; its cross-attention can access permitted source positions.

Before any hyperparameter search, inspect decoded inputs and targets from the actual training loader, not just the original text files. Confirm special tokens, truncation, document boundaries and padding masks. Manually inspect at least one very short example where every prediction is understandable.

CHECK: If you cannot explain exactly what token is predicted at each position, do not interpret the loss curve yet.

Loss is an average over something

Cross-entropy for one target token y is -log p(y). A dataset loss should normally be the sum of valid token losses divided by the number of valid target tokens. Averaging batch means without weighting is wrong when batches contain different valid-token counts. The same issue appears with variable-length gradient accumulation.

FORMULA: L = sum over valid targets of [-log p(y | context)] / number of valid targets. Perplexity = exp(L), when the logarithm is natural.

If two batches contain 100 and 900 valid tokens, their means 4.0 and 2.0 produce a token-weighted loss of 2.2, not 3.0. This difference can be large enough to reverse an experiment ranking.

Tokenization changes the unit of prediction. A character model's perplexity is not directly comparable with a subword model's perplexity. Report the tokenizer, vocabulary and normalization rules alongside loss. For cross-tokenizer comparisons, use a carefully defined common unit such as bits per byte, with identical text and explicit treatment of special tokens.

EXERCISE: Calculate the weighted loss for 200 tokens at 3.0 and 800 at 2.5. Answer: 2.6.

Split before you create near-duplicates

Randomly dividing overlapping token windows can put almost identical passages in both training and validation. The resulting validation score tests memorization of nearby text, not the intended generalization. Split at the source-document level before chunking, and deduplicate where appropriate. For time-dependent tasks, a chronological split may better match deployment.

Keep training, validation and final test roles separate. Training updates weights. Validation selects settings and checkpoints. The test set estimates performance after the selection process is complete. Repeatedly choosing configurations based on test results turns that set into another validation set.

A split should also represent the questions you care about. If your model translates one language pair, evaluate sentence lengths, rare words and named entities separately. An excellent average can hide a serious failure on long inputs.

DECISION: Spend your first debugging hour checking split construction rather than searching a new optimizer. Data leakage can make every hyperparameter conclusion look stronger than it is.

WORKSHEET: Record the split unit, deduplication procedure, source distribution, date boundary if any and number of valid tokens per split.

Vocabulary is an architecture choice

An embedding matrix contains V times d parameters, where V is vocabulary size and d is model width. A large vocabulary can dominate a small model's parameter budget. It also changes sequence lengths: more subword merges can shorten text sequences, but rare tokens receive fewer training examples.

Weight tying shares the input embedding and output projection when their shapes and semantics permit it. With tied weights, count the shared matrix once. With untied weights, the output projection adds roughly another Vd parameters, plus any output bias. Weight tying changes the model and should be recorded, not treated as an invisible implementation trick.

EXAMPLE: V = 32,000 and d = 512 require 16,384,000 embedding parameters. V = 8,000 at the same width requires 4,096,000. The difference can fund several small Transformer blocks, but the token sequences and prediction task also change.

Do not select vocabulary solely to hit an attractive parameter count. Measure tokenized lengths, unknown-token behavior where applicable, language coverage and downstream quality. Keep the tokenizer fixed during the initial architecture sweep so the losses remain comparable.

Worked data audit

A student reports that a 20M-parameter model achieves extremely low validation loss after one epoch. Their data pipeline first creates windows of 256 tokens with stride 1, then randomly assigns windows to train and validation. Most validation windows share nearly all their tokens with training windows.

The correct next step is to rebuild the split at document level and rerun a baseline. The previous validation numbers cannot justify choosing the model size or learning rate. They answer a contaminated question. Do not try to compensate by adding dropout while keeping the same evaluation setup.

Now suppose the corrected validation set is much harder because it contains a different topic. This may be an honest deployment scenario or an accidental mismatch. Inspect source distributions and decide explicitly which generalization target you want. Do not silently mix these two interpretations.

ANSWER: An honest baseline with a worse score is more useful than a leaked baseline with an impressive score. Record both the split change and the reason; do not present the old and new scores as a controlled hyperparameter comparison.

The symbols must mean something concrete

Use L for the number of Transformer blocks, d for residual-stream width, h for query heads, d_h for head dimension, f for feed-forward hidden width, T for context length and V for vocabulary size. In standard multi-head attention with equal-sized heads, d = h times d_h. This relationship is an implementation constraint for that architecture, not a universal law of all attention variants.

A standard decoder block contains causal self-attention and a position-wise feed-forward network, plus residual paths and normalization. An encoder block uses non-causal self-attention. A conventional encoder-decoder decoder also contains cross-attention to encoder outputs. Therefore, a decoder block in a translation model is not parameter-equivalent to a decoder-only block of the same width.

The residual stream is the representation carried between blocks. Heads split projected attention features; they do not normally create h independent full-width models. Increasing h while keeping d fixed makes each head narrower.

CHECK: For d = 384 and h = 6, d_h = 64. For d = 384 and h = 8, d_h = 48. Both satisfy divisibility; neither is automatically superior.

Count standard attention correctly

With dense projections of shape d by d, the query, key, value and output matrices contribute approximately 4d squared parameters. Biases add smaller linear terms if enabled. Splitting those projections into different numbers of heads does not change their total matrix size when d is fixed.

FORMULA: standard multi-head attention parameters approximately 4d^2. A two-matrix feed-forward network contributes approximately 2df. With f = 4d, a decoder-only block is approximately 12d^2 parameters.

For d = 512, attention contributes 1,048,576 weights and an f = 2,048 feed-forward network contributes 2,097,152. Total main block weights are 3,145,728. LayerNorm parameters and biases are excluded from this approximation.

This count assumes ordinary multi-head attention. Grouped-query attention reduces key and value projection widths and changes the count. A gated feed-forward network has three major matrices rather than two. Learn the structural formula instead of applying 12Ld squared to every model you encounter.

EXERCISE: What happens to main block weights if width doubles while f/d stays fixed? Answer: approximately four times as many.

From a block to a whole model

For a tied-embedding decoder with standard attention and f = 4d, a useful first estimate is N approximately Vd + 12Ld^2. Learned absolute position embeddings add Td. Normalizations, biases and implementation-specific modules add further parameters. Count actual trainable tensors to confirm the estimate.

EXAMPLE: V = 16,000, d = 384 and L = 8. Embeddings contribute 6,144,000. Blocks contribute 8 times 12 times 384 squared = 14,155,776. The main-weight estimate is 20,299,776. Learned positions at T = 512 would add 196,608.

This distinction matters when you describe a model as 20M parameters. Is that trainable parameters, all stored parameters, or only non-embedding parameters? Are shared tensors counted once? For frozen-backbone fine-tuning, total and trainable counts can be very different.

PRACTICE: Implement sum(p.numel() for p in model.parameters()) and a separate sum restricted to p.requires_grad. Check how your framework exposes shared tensors before relying on a custom counting routine.

DECISION: Reject any candidate that exceeds the budget after actual instantiation, even if a rounded estimate looked acceptable.

Shapes are a debugging tool

For batch B and sequence length T, hidden states usually have shape [B, T, d]. Query tensors can be viewed as [B, h, T, d_h]. Attention logits then have shape [B, h, T, T] in a straightforward implementation. The output is concatenated across heads and projected back to width d.

For cross-attention, query length is the target length T_t and key/value length is source length T_s. The attention-logit shape is [B, h, T_t, T_s]. A source padding mask must exclude padded keys. A decoder causal mask does not replace this source mask.

Write these shapes beside your code when debugging. A reshape that preserves the number of elements can still put dimensions in the wrong order. A model may run without crashing while mixing positions or heads incorrectly.

EXERCISE: With B = 2, h = 8 and T = 256, the straightforward self-attention score tensor has 1,048,576 elements. At two bytes per element it is 2 MiB for this tensor alone. Backpropagation needs more than this single allocation.

Worked comparison: heads versus width

Configuration A has d = 512, h = 8 and L = 8. Configuration B has d = 512, h = 16 and L = 8. Configuration C has d = 1,024, h = 16 and L = 8. All use f = 4d and tied embeddings.

A and B have the same main parameter count under standard attention. Their head dimensions are 64 and 32. They differ in how attention computation is partitioned and may differ in quality, kernel efficiency and intermediate memory. C keeps head dimension 64 but doubles the residual width, making the main block parameters four times larger and the embeddings twice as large.

ANSWER: Saying that B and C both have sixteen heads conceals a huge architectural difference. Head count alone is not a useful model-size description. Report at least L, d, h, f, vocabulary, context and attention variant.

NEXT EXPERIMENT: Compare A with B to study head partitioning at fixed width. Compare A with C only when you intend to change capacity and cost. Do not interpret either comparison as proving that sixteen heads are universally better.

Three budgets, three different bottlenecks

Parameter count, training FLOPs and peak memory are related but not interchangeable. A model can fit in memory and still take too long to train. A small model with very long sequences can have expensive attention. A frozen model can have few trainable parameters yet still require substantial memory for activations and base weights.

Separate persistent memory from temporary memory. Persistent allocations include parameters, optimizer states and often gradients. Temporary allocations include saved activations, attention intermediates, kernel workspaces and allocator overhead. Exact totals depend on dtype, optimizer, kernels, checkpointing and distributed strategy.

A conventional mixed-precision Adam training estimate is sometimes around 16 bytes per parameter when it includes low-precision weights and gradients, FP32 master weights and two FP32 moment tensors. This is an illustrative accounting model, not a guarantee; other implementations store different combinations.

DECISION: Use a symbolic estimate to reject impossible candidates, then run a short measured forward/backward/update benchmark. Include optimizer initialization and a safety margin. Memory measured before the first optimizer step may omit lazily allocated optimizer states.

Estimate training compute without worshipping it

A common rough estimate for dense Transformer training is C approximately 6ND, where N is a relevant parameter count and D is the number of training tokens. It captures dominant parameter-matrix work in regimes where that approximation is appropriate. It is not an exact cost model, and attention's sequence-length-dependent work can matter significantly.

FORMULA: approximate training FLOPs = 6 times N times D. Estimated wall time = FLOPs / achieved training FLOPs per second.

For N = 80 million and D = 1 billion, this estimate gives 4.8 times 10^17 FLOPs. Dividing by advertised GPU peak throughput is usually optimistic. Real throughput depends on utilization, memory bandwidth, kernels, batch shapes and data loading. Prefer measured tokens per second from your actual configuration.

If your measured sustained throughput is 20,000 tokens per second, one billion processed tokens takes about 50,000 seconds, or 13.9 hours, excluding interruptions and extra evaluation overhead. That is an illustrative calculation, not a prediction for a named GPU.

CHECK: Always say whether D counts unique corpus tokens or repeated token exposures.

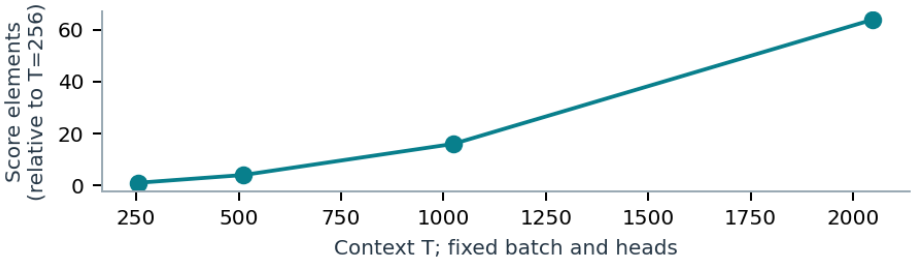
Understand the sequence-length cost

For a straightforward attention implementation, score storage scales with B h T squared, and attention matrix multiplication work scales roughly with B T squared d. Projection and feed-forward work scale roughly with B T d squared. Which term dominates depends on width, context and implementation.

Doubling T at fixed B approximately quadruples the score-matrix elements and doubles the number of processed tokens. If you halve B to keep tokens per batch constant, score storage still roughly doubles. This explains why fitting a short-context model says little about whether the same configuration fits a much longer context.

Memory-efficient attention kernels can avoid materializing the full score matrix and reduce memory traffic. They do not make dense attention's mathematical work independent of sequence length. Activation checkpointing saves activation memory by recomputing parts of the forward pass during backward.

DECISION: If long context is required by the task, budget for it early. If it is merely an attractive number, measure whether longer contexts improve relevant validation examples before paying the cost.



Reserve compute for learning from experiments

A practical project budget might allocate a small portion to correctness checks, a larger portion to short optimization pilots, most of the remainder to finalists, and a reserve for recovery. There is no universal percentage. The key is that the final run must not consume resources you have already promised to comparison runs.

Short runs answer limited questions. They can reveal divergence, severe underutilization or obvious optimizer failures. They may rank architectures incorrectly if learning curves cross later. A small pilot that undertrains every large model can unfairly favor the smallest candidate.

Use at least two views of efficiency: validation loss against processed tokens and validation loss against wall-clock time. Token efficiency matters for limited data; time efficiency matters for limited compute. Report peak memory as a third axis when deployment or hardware constraints matter.

WORKSHEET: List remaining GPU-hours, a measured time per update, intended update counts, evaluation cost, expected failed runs and a recovery reserve. If the sum exceeds your budget, reduce the search before launching it.

Worked budget ledger

Assume a hypothetical machine sustains 8,000 tokens per second for your baseline. You have six training hours and reserve one hour for pilots and evaluation. Five hours of final training can process approximately 144 million tokens. If the corpus has 24 million tokens, that corresponds to about six passes, assuming all tokens are used each pass.

A larger candidate processes only 4,000 tokens per second. Under the same five-hour budget, it sees about 72 million tokens. Comparing the models after five hours is a valid time-budget comparison, but it is not a comparison at equal token exposure. Both questions are useful; they simply need different labels.

ANSWER: Do not promise a fixed model size first and hope the training budget follows. Measure throughput, decide the comparison budget and choose candidates that can receive meaningful training.

EXTENSION: If validation begins worsening after 80 million tokens for the smaller model, spending all 144 million is not automatically helpful. The best checkpoint and the planned compute budget are separate decisions.

Width is representational bandwidth

The residual width d controls the number of features available at each token position. Every block reads and writes this vector. Increasing width enlarges attention projections, feed-forward layers and embeddings. It can improve capacity, but it also changes optimization and hardware utilization.

Width is not the same as the number of concepts the model understands. Features can be distributed and superposed; there is no one-neuron-per-concept accounting rule. Think of width as a structural capacity and compute choice, not a semantic inventory.

At a fixed block count and fixed feed-forward ratio, block parameters scale approximately as d squared. Consequently, a modest width increase can be much more expensive than it first appears. Embedding parameters grow linearly with d, so very small models with large vocabularies have different cost proportions from large deep models.

DECISION: Choose a short list of hardware-friendly widths that fit your budget and support valid head dimensions. Then compare them using an explicit token or time budget, rather than sweeping every integer width.

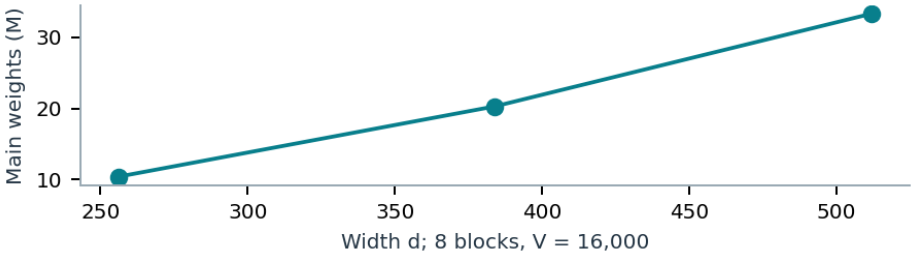
Work backward from a parameter budget

Using N approximately Vd + 12Ld squared for a conventional tied decoder, you can estimate feasible widths before coding. For a fixed L and V, solve the quadratic or enumerate a few convenient d values. Enumeration is often clearer because divisibility and kernel constraints already restrict useful candidates.

EXAMPLE: Let V = 16,000 and L = 8. At d = 256, the main count is 10.39M. At d = 384 it is 20.30M. At d = 512 it is 33.36M. These are estimates excluding normalization, biases and positional parameters.

Doubling width from 256 to 512 does not exactly quadruple total parameters because the embedding term only doubles. The block term does quadruple. When a source gives a scaling statement, identify which part of the model it describes.

CHECK: For each candidate, calculate head dimension and feed-forward width, instantiate the model, count actual tensors and measure peak memory. A parameter budget is a screening tool, not a complete feasibility certificate.



A controlled width experiment

Select two or three widths while keeping tokenizer, data split, context, normalization, attention type and feed-forward ratio fixed. Keep the intended comparison budget explicit. If you compare under equal wall time, record different token counts. If you compare under equal tokens, record different training times.

Learning rate may need retuning when width changes under ordinary parameterization. A single shared learning rate can make one width appear worse merely because it trains less effectively. Give each candidate a small, comparable learning-rate search rather than comparing a tuned baseline with an untuned alternative.

Record training loss, validation loss, throughput and peak memory. If the wider model fits training better but not validation, investigate data limits and overfitting. If it fits neither better, check optimization and training duration before concluding that width is useless.

PRACTICE: Predict how increasing d from 384 to 512 affects the main block term. Answer: a factor of (512/384)^2, approximately 1.78. Embeddings grow by 1.33, not 1.78.

When width is the wrong intervention

A model that cannot memorize one tiny batch may have a bug or an optimization problem. Increasing width before resolving that failure makes debugging harder. A model with good teacher-forced loss but poor free generation may suffer from decoding or task mismatch. Again, width is only one possible explanation.

Consider the deployment bottleneck too. A wider model can increase per-token matrix multiplication cost and KV-cache size. If you need low-latency generation on limited hardware, the best validation score at unlimited inference cost may be irrelevant.

Width and depth tradeoffs are empirical. Wider shallow models may exploit hardware parallelism well; deeper models add sequential transformations. Neither statement determines which model will win on your data. You must measure quality and cost jointly.

DECISION: Increase width when capacity appears limiting, optimization is functioning, enough data and training budget remain, and inference cost is acceptable. Write those conditions explicitly; do not infer all of them from a single disappointing sample.

Exercise: choose the next width

You compare d = 256 and d = 512 at equal processed tokens. The small model's evaluation-mode training loss is 3.4 and validation loss is 3.5. The large model's corresponding losses are 2.8 and 3.6. Both were given reasonable learning-rate pilots and neither is unstable.

The larger model has learned the training data better but has not improved validation. This is evidence against spending the next experiment only on additional width. It does not prove that every larger model is bad. Your training data, regularization, data distribution or checkpoint selection may be limiting generalization.

ANSWER: Preserve the smaller model as the better current validation baseline. Inspect the large model's best earlier checkpoint and evaluate whether its validation loss improved before worsening. Consider more clean data or a shorter horizon before expanding capacity again.

FOLLOW-UP: If the larger model also improves an important rare-token metric, the decision becomes multi-objective. Report that tradeoff instead of forcing all quality into one average. A professional decision can prefer a slightly worse average to satisfy a specific requirement.

Depth adds sequential computation

Each Transformer block applies another attention and feed-forward transformation to the residual stream. More blocks allow additional stages of computation and feature refinement. This is different from increasing the width of each stage. A deeper model is not simply a wider model written differently.

At fixed width and block design, the main block parameter count grows linearly with L. Training and inference also gain additional sequential work. GPUs can parallelize many operations within a block, but the next block depends on the previous block's output. Depth can therefore have a latency cost not captured by parameter count alone.

Residual connections and normalization help make deep models trainable, but they do not remove every optimization difficulty. Initialization, learning-rate schedule and architecture details become important as depth grows.

DECISION: Start with a depth you can train and debug quickly. Expand only after you know whether the baseline is capacity-limited, optimization-limited or data-limited. Six versus twelve blocks is a hypothesis about useful additional computation, not an upgrade with a guaranteed payoff.

Equal-parameter depth-width tradeoffs

Ignoring embeddings and small terms, keeping Ld squared constant approximately preserves block parameters when f/d is fixed. Doubling L therefore suggests reducing d by about the square root of two if you want a similar block budget. Vocabulary embeddings prevent this from being an exact whole-model equivalence.

FORMULA: L1 d1^2 approximately equals L2 d2^2. Thus d2 approximately d1 sqrt(L1/L2).

Suppose one model uses L = 8 and d = 512. A sixteen-block model with roughly the same block parameter budget would have d near 362. Choose a practical nearby width and then recount; do not force an awkward dimension just to satisfy the approximation.

Even equal-parameter candidates can have different FLOPs, activation memory, throughput and generation latency. Their learning-rate preferences can also differ. Parameter matching is one control, not a declaration that every other variable is equal.

EXERCISE: If depth triples at fixed width, what happens to embeddings? Answer: their count stays unchanged; only the repeated block contribution roughly triples.

Plan a depth sweep with enough training

A useful initial sweep might compare a shallow baseline, an intermediate model and one deeper model. The actual depths depend on your budget; numbers such as 4, 8 and 12 are illustrative candidates, not universal recommendations. Keep width and other block details fixed if the question is specifically about adding depth.

Run correctness checks on every construction, then provide a small learning-rate search with comparable effort. Train long enough to see whether early rankings persist. If the deep candidate starts more slowly, a very short pilot may be an unfair quality estimate even though it still measures short-budget usefulness.

Inspect per-layer gradient norms or update ratios when deep training fails. A global norm alone can hide layers receiving tiny or unusually large updates. Check whether all blocks participate in the forward pass and receive gradients.

DECISION: Increase depth when the additional quality justifies its measured training and inference cost. Keep the best current model until the new candidate earns its place under the chosen metric and budget.

Depth and stability interact

A loss spike after adding blocks is not proof that the new architecture is fundamentally unsuitable. The same peak learning rate and initialization may no longer be appropriate. Conversely, continually lowering learning rate can hide a poorly constructed residual or normalization path without producing useful learning.

For a small custom implementation, favor a well-understood block layout before experimenting with novel ordering. Pre-normalization places normalization inside the residual branch before the sublayer; post-normalization normalizes after the residual addition. Their optimization behavior can differ. Treat the choice as part of the architecture, not a cosmetic refactor.

A useful debug sequence is: verify finite activations and gradients, inspect layerwise statistics, test a lower peak learning rate, examine warmup, then investigate initialization and block equations. If only certain batches trigger the failure, inspect those batches before changing depth again.

CHECK: Log the block count from the actual module list and verify every intended block is called. Accidental weight sharing can make twelve applications contain only one block's trainable parameters.

Worked choice: deeper or longer training?

An eight-block model is still improving steadily when its run ends. You have enough compute either to train it longer or to launch a sixteen-block model for half as many tokens. There is no automatic reason to choose the larger architecture. The current curve suggests that undertraining may be an important limitation.

First estimate the expected gain from extending the baseline using its recent validation slope and schedule state. Then run a small deeper-model pilot if affordable. A new model needs optimization checks and may spend much of its budget in a less effective early phase.

ANSWER: Under a tight budget, completing a promising baseline can be more valuable than replacing it with an undertrained larger model. This is a resource-allocation judgment, not a claim that shallow models are inherently better.

REPORT: Include the token counts, learning-rate trajectories and wall times. If the larger model loses at half the tokens, say it lost under that compute plan. Do not generalize the result into a statement about maximum achievable quality.

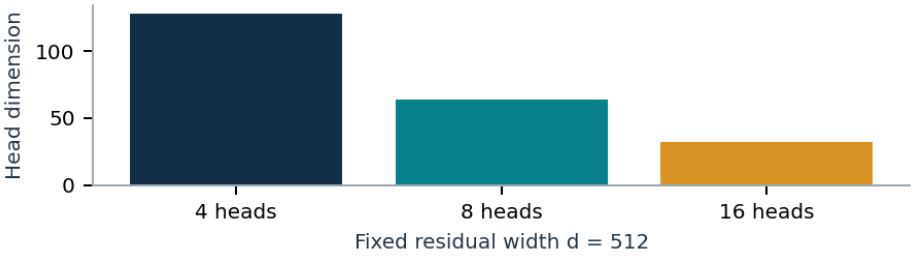
Heads divide a fixed attention representation

In ordinary multi-head attention, each head forms its own attention pattern using projected queries and keys, then combines projected values. The concatenated head outputs return to residual width d through an output projection. At fixed d, more heads mean smaller d_h, not more total residual features.

The original Transformer popularized a particular head configuration, but copying its head count without its width changes the head dimension. A rule such as always use eight heads ignores the quantity that each head actually operates on. Start by considering both d and d_h, then derive h when the architecture requires d = h d_h.

Head dimensions such as 32, 64 or 128 can be convenient candidates in some implementations. They are examples for a search space, not biological constants or guaranteed optima. Kernel support, model size and task can change what is sensible.

DECISION: Use a valid, efficient baseline and test only a few alternatives. Head count is normally a second-stage refinement after data, optimization, width and depth are credible.



Why attention scores are scaled

For one head, logits are QK transpose divided by sqrt(d_h), followed by masking and softmax. If independent query and key components have roughly unit variance, their dot product has variance growing with d_h. Dividing by sqrt(d_h) keeps the scale more controlled under those assumptions.

FORMULA: Attention(Q,K,V) = softmax(QK^T / sqrt(d_h) + mask) V.

The variance argument motivates the scaling; actual trained activations need not satisfy the simplifying assumptions exactly. The scaling does not imply that every head dimension has identical expressiveness or optimization behavior. Narrower heads have less feature dimension for their query-key comparisons, while additional heads allow more separately normalized attention patterns.

CHECK: In a custom implementation, scale by the dimension used in each query-key dot product, not by the full model width merely because it is easier to access. If your fused attention function already applies scaling, do not apply it again unintentionally.

EXERCISE: d = 512, h = 8 gives scaling 1/8. With h = 16, d_h = 32 and scaling is 1/sqrt(32).

Design a head-count experiment

For a model with d = 384, reasonable illustrative candidates include h = 6, 8 and 12, producing head dimensions 64, 48 and 32. Keep width, depth, feed-forward width, tokenizer, context and training budget fixed. This isolates head partitioning more cleanly than changing heads and width together.

Check actual throughput and memory. In straightforward attention implementations, increasing h at fixed d can enlarge the explicitly stored attention-score tensor B h T squared, even though dominant projection parameter counts remain constant. Fused kernels can change the memory behavior and performance ranking.

Use the same primary validation metric and evaluate long-context examples separately if those are important. An apparent benefit on one sample or one visually appealing attention map is insufficient. Attention maps can be useful diagnostics but do not automatically establish a faithful explanation of the model's decision.

STOP RULE: If the candidates perform within the noise and one is faster or simpler, retain that candidate. You do not need to discover a globally perfect head count to make a sound engineering choice.

MHA, MQA and GQA are different designs

Multi-head attention normally gives each query head its own key and value head. Multi-query attention shares one key and one value head across query heads. Grouped-query attention uses an intermediate number of key/value heads, shared by groups of query heads. These designs affect parameter counts, KV-cache size and inference behavior.

If there are h_q query heads and h_kv key/value heads of dimension d_h, standard dense projection weights are approximately 2d(h_q d_h) + 2d(h_kv d_h), including query and output plus key and value projections. When h_q d_h = d, this becomes 2d^2 + 2d h_kv d_h.

Reducing KV heads can reduce inference cache memory substantially, but the quality and speed tradeoff must be tested. Retrofitting a trained standard-attention model is not generally equivalent to changing one integer and continuing unchanged.

DECISION: For your first from-scratch student baseline, standard MHA is easier to reason about. Study GQA when inference memory or serving throughput becomes an explicit constraint, and count query heads separately from KV heads.

Worked head-count answer

You ask: my model width is 256; should I use eight or sixteen heads? Eight heads gives d_h = 32 and sixteen gives d_h = 16. Neither answer follows from width alone. Sixteen heads does not double the standard attention parameters, and sixteen-dimensional heads may or may not be adequate for your task.

A defensible plan is to start with a convenient configuration, perhaps four or eight heads, verify learning, and compare a small set at fixed width. Include the sixteen-head option only if you have a reason and budget to test it. Check whether your attention kernel handles the shapes efficiently.

ANSWER: State the assumption, not a universal prescription: for this small baseline, I choose eight heads because it gives a valid 32-dimensional head and measured acceptable throughput; I will revisit it after the optimizer and capacity experiments. That is already much stronger reasoning than choosing sixteen because it sounds more powerful.

REFLECTION: What result would make you change? A reproducible validation improvement or a useful memory/latency advantage, not merely a more complicated attention visualization.

Attention is not the whole Transformer

The feed-forward network applies a nonlinear transformation independently at each token position. Attention mixes information across positions; the feed-forward network transforms the features at each position. Both are essential parts of the block's capacity and cost.

For an ordinary two-matrix network, a width-d vector is projected to f, passed through an activation, and projected back to d. Ignoring biases, its parameter count is 2df. The common illustrative ratio f/d = 4 gives 8d squared parameters, often more than the attention projections in the same block.

If you change the number of attention heads but ignore feed-forward width, you may be tuning a relatively small structural detail while leaving a major capacity allocation unexamined. However, changing f also increases activation memory and computation, so larger is not free.

DECISION: Record f explicitly. A model description containing only blocks and heads is incomplete. Use a conventional feed-forward design first, then investigate expansion ratio if quality or efficiency suggests a need.

Gated networks need different accounting

A gated feed-forward network such as SwiGLU uses two input projections whose outputs interact multiplicatively, followed by an output projection. Its main parameter count is approximately 3df rather than 2df. Comparing ordinary and gated networks at the same f gives the gated network more parameters.

For roughly equal feed-forward parameters, choose f_gated approximately two-thirds of f_plain. If a plain network uses f_plain = 4d, the matched gated width is approximately 8d/3, rounded to a convenient supported value. This is a parameter-matching calculation, not a claim of exact compute or quality equivalence.

EXAMPLE: At d = 384, a plain width of 1,536 gives 1,179,648 main feed-forward weights. A gated width of 1,024 gives the same main weight count: 3 times 384 times 1,024.

CHECK: When reading a configuration from another project, identify the activation and number of projections before using its intermediate width. A number copied without the block equation can silently change your parameter budget by fifty percent in this component.

Choose the expansion ratio empirically

An initial study might compare f/d values of 2, 3 and 4 for a plain feed-forward network. These are illustrative search points. Hold attention, depth, width, data and optimizer protocol fixed. Count total parameters and report cost, because this is a capacity experiment rather than a free substitution.

If the larger feed-forward layer improves training loss but not validation, the extra capacity may be overfitting or may need more appropriate data and regularization. If a narrower feed-forward layer preserves validation quality while improving speed, it is a strong efficiency candidate.

Avoid sweeping activation type, expansion ratio, dropout and learning rate simultaneously without a structured design. You will not know which change caused the result. A modest optimizer retune for each architecture is sensible, but document its search budget.

PRACTICE: Estimate the parameter reduction from changing f from 4d to 2d. The feed-forward term halves from 8d squared to 4d squared, while standard attention stays at 4d squared. Main block parameters drop from 12d squared to 8d squared, a one-third reduction.

Activation changes are not cosmetic

ReLU, GELU and SiLU-based gated variants differ in smoothness, gating and output statistics. A reasonable choice often follows a well-established architecture family rather than a fresh activation search. Use an existing coherent recipe as a baseline and alter one component with a clear question.

Initialization and normalization interact with activation statistics. A custom activation with a radically different output scale can destabilize a previously stable block. If you change it, inspect activation means, variances and gradient norms across layers during a short pilot.

The best activation in a paper may have been tested at a different scale, data distribution and optimizer setting. The paper offers evidence, not a guarantee for your project. Reproduce the relevant comparison conditions when possible, or explicitly label your experiment as an adaptation.

DECISION: Spend activation-search compute only after the baseline is trustworthy. For a learning project, implementing and understanding one plain and one gated network is more valuable than trying ten names without controlling parameter count.

Exercise: fair versus unfair comparison

Model A uses a plain feed-forward network with d = 512 and f = 2,048. Model B uses a three-matrix gated network with the same d and f. A's feed-forward main weights are 2,097,152; B's are 3,145,728. B has fifty percent more feed-forward weights, though not fifty percent more total model weights.

If B wins, you cannot attribute the entire gain to gating alone. It also received more parameters and computation. To study gating under roughly matched feed-forward parameter count, choose a gated f near 1,365, then round to a practical value such as 1,344 or 1,408 and report the resulting difference.

ANSWER: There are two legitimate experiments: compare conventional recipes as deployed, or compare architectural mechanisms under a controlled budget. State which one you are doing. Neither requires pretending that all costs are identical.

FOLLOW-UP: Measure wall time even in the matched-parameter comparison. Different operation shapes and elementwise gates can change throughput despite similar matrix parameter totals.

Context length is a task requirement

The context window limits how much preceding or source information can be available to a prediction. A long window helps only when useful information exists at long distances and the model learns to use it. Many tasks have mostly local dependencies; others genuinely require document-level context.

Inspect your data's token-length distribution before choosing T. Determine what fraction of examples are truncated at each candidate length and whether the removed information matters. For translation, inspect source and target lengths separately. A shared maximum length can hide asymmetric requirements.

Training context and inference context are not automatically interchangeable. A model with a positional scheme that accepts longer inputs may still perform poorly beyond the lengths seen during training. Mechanical compatibility is not evidence of reliable extrapolation.

DECISION: Choose the smallest context that adequately covers the target task, then test longer windows on a deliberately constructed long-dependency evaluation. Do not pay quadratic attention costs solely to advertise a large context number.

Position methods change the assumptions

Learned absolute position embeddings add a trainable vector indexed by position. Sinusoidal position encodings use fixed functions. Rotary position embeddings transform queries and keys using position-dependent rotations. Relative-bias approaches modify attention scores based on relative positions. These are different mechanisms with different extrapolation behavior.

For learned absolute embeddings, extending beyond the trained position table requires a design decision such as resizing or interpolation. For rotary approaches, accepting a larger position index is easy, but preserving quality can require an appropriate extension method and validation. Avoid making a generic claim that rotary embeddings solve long-context generalization.

Position settings such as rotary base or scaling factors are advanced hyperparameters. Begin with a coherent baseline implementation and test the intended context range. Changing several positional settings while also extending T makes attribution difficult.

CHECK: Verify position IDs when packing documents, padding on different sides or resuming generation with a cache. A positional-index bug can look like a model-capacity or learning-rate problem.

A context sweep has hidden confounders

Suppose you increase T from 256 to 1,024 while leaving sequence batch size unchanged. Tokens per update quadruple, optimizer updates per token change, memory requirements rise and the model sees longer dependencies. You have changed several aspects of training at once.

For a cleaner context comparison, try to keep valid tokens per update and total processed tokens comparable. Adjust microbatch size or accumulation accordingly. Even then, the arrangement of sequences and available conditioning differs, so the objective itself has changed in a meaningful way.

Evaluate loss by position and by examples that need distant information. An average dominated by short local predictions can conceal the benefit or failure of long context. Also measure throughput, because a context gain that costs too much may not be useful under the final budget.

EXERCISE: At a token batch of 8,192, T = 256 allows 32 full sequences, while T = 1,024 allows eight. With padding or variable lengths, these counts no longer describe valid tokens exactly.

Packing and boundaries matter

Packing concatenates shorter sequences to reduce padding waste. Whether tokens may attend across document boundaries depends on the intended training objective. Simple language-model pretraining often concatenates documents with a separator; stricter packing can use boundary-aware attention masks. These choices are not equivalent.

If you pack unrelated translation examples, the encoder and decoder masks must preserve the intended pairing. Accidental cross-example attention can leak information and invalidate training or evaluation. Begin with a simpler correct pipeline before adding complicated packing.

Track padding fraction and valid-token throughput. A speed comparison based on total padded tensor elements can reward a wasteful pipeline. For gradient accumulation, normalize by valid target tokens when lengths differ materially.

DECISION: Optimize packing after correctness is established, and keep its behavior fixed during initial architecture comparisons. Save the packing rules with the data configuration so a resumed run does not silently train on a different objective.

CHECK: Inspect a packed example visually, including boundary tokens, positions, attention permissions and target-loss masks.

Worked context decision

Your corpus has a median length of 180 tokens and a 95th percentile of 700. You consider T = 256 and T = 1,024. The shorter context is faster, but it truncates important conclusions in some documents. The longer context uses more memory and reduces throughput.

A sensible evaluation includes ordinary average loss plus a subset where a target depends on information beyond 256 tokens. Compare both contexts with controlled token budgets and report their time cost. You might choose the shorter context for a constrained baseline and the longer context for a document-completion goal.

ANSWER: The length distribution alone does not decide. It tells you how often truncation occurs; task analysis tells you whether truncation destroys useful information. The final choice combines that evidence with measured quality and cost.

EXTENSION: A length curriculum can begin with shorter sequences and later use longer ones, but it changes the training distribution and schedule. Treat it as a distinct experiment with its own token accounting, not a free shortcut guaranteed to match full-length training.

Translation has two information pathways

An encoder builds source representations using self-attention over permitted source tokens. A decoder predicts the target autoregressively using causal self-attention and cross-attention to encoder outputs. The encoder and decoder solve related but different subproblems, so their layer counts need not be equal.

A source representation can be computed once per input. The decoder participates repeatedly during autoregressive generation, usually with caches. Adding decoder depth can therefore affect generation latency differently from adding encoder depth. Training parallelizes target positions under teacher forcing, so training speed alone does not reveal serving cost.

For a first translation baseline, a balanced, conventional architecture is easier to debug. Once it works, compare encoder-heavy and decoder-heavy allocations under an explicit budget and real generation measurements.

DECISION: Report encoder depth L_e and decoder depth L_d separately. The phrase six-layer Transformer is ambiguous unless you specify whether it means six total blocks, six per stack or something else.

Count cross-attention too

Under equal source and target representation width d, a standard encoder block with f = 4d has about 12d squared main weights. A conventional decoder block adds cross-attention, giving about 16d squared: 4d squared self-attention, 4d squared cross-attention and 8d squared feed-forward.

FORMULA: main stack weights approximately 12L_e d^2 + 16L_d d^2. Add source and target embeddings, output projection if untied, positions, biases and normalizations as appropriate.

If source and target widths differ, cross-attention projection shapes change. If the vocabularies are shared and embeddings are tied, count the shared matrices once. If they are separate, count each explicitly. Never copy the decoder-only 12Ld squared estimate without checking the block structure.

EXAMPLE: At d = 256, moving one block from decoder to encoder saves about 4d squared = 262,144 main weights under these assumptions. It also changes the computation pathway, so equal total blocks is not equal model design.

Allocate depth around the bottleneck

A translation model can fail because source representations are inadequate, because target language modeling is weak, because alignment is difficult or because the data itself is noisy. Increasing both stacks together can improve quality but does not identify the bottleneck.

Use targeted diagnostics. Examine errors involving source entities, omissions, word order and grammatical target fluency. Compare teacher-forced validation loss with actual generated translations. A decoder can have decent next-token loss while making early generation mistakes that change later context.

A small architecture study might keep total approximate stack parameters similar while comparing balanced, encoder-heavy and decoder-heavy designs. Retune peak learning rate modestly and measure both translation quality and end-to-end latency. Keep beam search and length settings fixed during this architecture comparison.

DECISION: Prefer the allocation supported by the task and deployment evidence. More encoder layers are not universally better for translation, and more decoder layers are not universally better because generation happens there.

Teacher forcing and decoding are different tests

During teacher-forced training, the decoder receives the true previous target tokens. During generation, it receives its own previous outputs. An early mistake can change all later predictions. This mismatch helps explain why training loss is not a complete description of generated quality.

Use both token-level validation and a fixed generation evaluation. Freeze decoding settings while comparing training hyperparameters. Otherwise a beam-size or length-penalty change may be mistaken for an architecture improvement. Include representative human inspection rather than relying on one metric alone.

Label smoothing, when used, changes the training objective. The reported smoothed training loss is not directly the same quantity as ordinary validation negative log-likelihood. Document which loss is being plotted and avoid diagnosing a train-validation gap from mismatched objectives.

CHECK: Ensure the decoder input is shifted, future target tokens are masked, source padding is masked and end-of-sequence handling is correct. These implementation details can dominate the effect of choosing four versus six layers.

Worked translation baseline

Consider a hypothetical student translation project with a shared 16,000-token vocabulary, width 256, four encoder blocks and four decoder blocks. Use a plain f = 1,024 network and a valid attention head partition. If the shared embedding is also tied to the output projection, its main count is 4,096,000.

The encoder stack contributes 4 times 12 times 256 squared = 3,145,728. The decoder stack contributes 4 times 16 times 256 squared = 4,194,304. Total main weights are about 11.44M before smaller terms and positions. Untied embeddings or output weights change this substantially.

ANSWER: This is a calculable baseline, not a prescription for your actual corpus. First verify a tiny-batch overfit and masking, then tune learning rate and training duration. Compare larger stacks only after those checks.

NEXT STUDY: Evaluate a six-encoder/two-decoder design and a two-encoder/six-decoder design with recounted budgets. Measure generation latency and translation errors. Do not assume equal total block count makes their parameter counts equal.

A stable starting scale matters

Initialization determines the early distribution of activations and gradients. If signals grow rapidly across layers, training can overflow or become unstable. If signals collapse, updates may become ineffective. Residual connections help information flow but also accumulate contributions from many blocks.

Use an initialization scheme consistent with the architecture you implement. Copying only the optimizer from a published model while changing normalization and residual initialization can produce a different stability regime. Framework defaults are useful, but you should know where they are applied.

Inspect a forward and backward pass before a long run. Check finite values, activation magnitudes and gradient magnitudes at several depths. You do not need every layer to have identical statistics; you need to notice extreme growth, collapse or missing gradients.

DECISION: For a learning project, begin with a standard coherent block recipe. Treat experimental initialization changes as separate interventions once the baseline is correct. A smaller learning rate can reduce symptoms without fixing a badly scaled computation graph.

Pre-norm versus post-norm

A pre-norm residual sublayer can be written x_next = x + F(Norm(x)). A post-norm version can be written x_next = Norm(x + F(x)). Their equations differ in where the identity path and normalization act. This affects gradients and training behavior, particularly as depth grows.

Pre-norm layouts are a practical baseline for many modern language-model implementations, but that does not make every pre-norm configuration stable or every post-norm configuration bad. The complete recipe includes initialization, optimizer, learning-rate schedule and possibly residual scaling.

LayerNorm typically normalizes using a mean and variance across features. RMSNorm uses a root-mean-square scale without the same mean-centering operation. Their parameter counts and behavior differ. They are architecture choices, not names to substitute without checking the equations.

CHECK: Normalization epsilon is a numerical-stability parameter. An extreme value can change behavior. Start with a conventional implementation value and only tune it when you have a specific numerical or optimization reason.

Residual scaling is a hypothesis

Some deep-network recipes scale residual branch initialization or outputs to control accumulated variance. The correct factor depends on the architecture and derivation. A factor copied from a different block layout may not have the intended effect.

For a student implementation, first compare your code with the chosen reference equations. Check whether residual dropout occurs before addition and whether normalization is applied once or accidentally twice. Verify that all residual tensors have the intended shape and dtype.

If deeper candidates become unstable while shallow ones work, inspect layerwise activation growth and update norms. This gives a reason to investigate residual scaling rather than blindly trying arbitrary constants. Keep a conventional baseline so you can tell whether the new scheme helps.

EXERCISE: If every residual branch is initialized to produce very large outputs, why might a small global gradient clip not solve the whole problem? Answer: clipping bounds a gradient norm during updates; it does not directly repair the forward-pass activation scale or the architecture's information flow.

The tiny-batch overfit test

Take a very small fixed batch, disable unnecessary augmentation and usually dropout for this diagnostic, and repeatedly optimize it. A sufficiently capable model should drive the training objective much lower. This test checks whether the forward pass, labels, masks, loss and gradients can cooperate at all.

Failure is a debugging signal, not automatic proof of insufficient capacity. Check that parameters require gradients, the optimizer contains them, gradients are not accidentally detached, the learning rate is nonzero and the target shift is correct. Inspect actual parameter changes after an update.

Passing the test does not prove generalization, absence of leakage or correctness on every input. It is a necessary practical sanity check, not a complete test suite. Restore normal training settings afterward and record that the diagnostic used a special configuration.

DECISION: Run this test before expensive architecture sweeps. It is far cheaper to discover a masking bug on a single batch than after comparing eight learning rates for several hours each.

Worked initialization diagnosis

A twelve-block model produces finite loss on its first batch but non-finite values after a few updates. A four-block version works. Layerwise logs show activation magnitudes already growing strongly with depth before optimization. Lowering the learning rate delays failure but does not remove this pattern.

This evidence points toward a forward-scale or block-implementation issue in addition to possible optimizer sensitivity. Verify normalization placement, residual additions and initialization. Check for an accidental repeated multiplication or duplicated residual addition. Compare with the intended architecture equation on a tiny input.

ANSWER: Do not conclude that twelve layers are too many for the dataset. The failure occurs before data-driven generalization can be assessed. Repair the numerical and architectural issue, then rerun the controlled comparison.

FOLLOW-UP: Once stable, log activation statistics again. The goal is not to force arbitrary unit variance everywhere, but to establish a reasonable, reproducible baseline in which gradients and updates remain finite and useful.

Learning rate acts on an update direction

For simple gradient descent, theta_next = theta - eta times gradient. The learning rate eta controls step size, while the gradient gives a local direction of increasing loss. Stochastic training uses a batch estimate of that gradient, so directions vary between updates.

The same numerical eta can behave very differently under different optimizers. Adam rescales coordinates using running statistics; SGD uses a different update rule. Therefore, an Adam learning rate of 0.001 is not meaningfully equivalent to an SGD learning rate of 0.001.

Loss surfaces are not globally simple bowls. A rate that works in one phase can be unstable or inefficient in another. Schedules change eta over time to trade rapid progress against finer adjustment, but no schedule guarantees the best path for every model.

DECISION: Think in terms of actual parameter updates, not the learning-rate number alone. Monitor loss, gradient norms and, when helpful, update-to-parameter ratios. An optimizer setting is a mechanism, not a badge of sophistication.

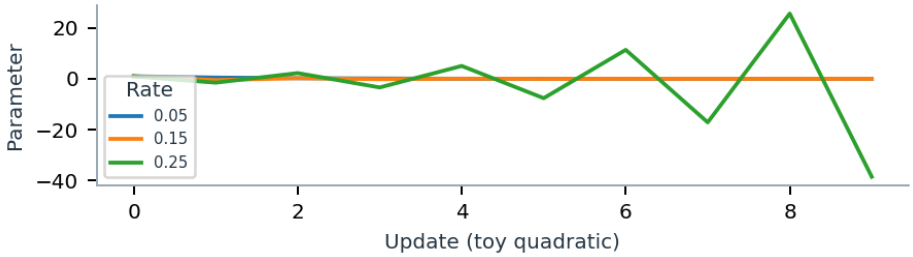
A quadratic makes stability visible

Consider L(theta) = 0.5 a theta squared with a positive. Its gradient is a theta, so gradient descent gives theta_next = (1 - eta a) theta. Convergence requires the magnitude of 1 - eta a to be less than one, giving 0 < eta < 2/a.

If eta a is small, convergence is slow. If eta a lies between one and two, iterates oscillate while shrinking. If eta a exceeds two, they diverge. This toy example explains why step size must respect curvature, but neural networks and adaptive optimizers are more complicated.

EXAMPLE: With a = 10, eta = 0.05 multiplies theta by 0.5 each step. Eta = 0.15 multiplies it by -0.5. Eta = 0.25 multiplies it by -1.5 and diverges.

LESSON: A loss that oscillates can indicate an overly aggressive update, but stochastic batches also create noise. Inspect a trend and the relevant statistics before diagnosing every upward step as instability.



Adam's moments in plain language

Adam maintains an exponential moving average of gradients and an exponential moving average of squared gradients. The first captures a smoothed direction; the second estimates a scale for each coordinate. Bias correction compensates for the moving averages starting at zero.

FORMULAS: m_t = beta1 m_(t-1) + (1-beta1) g_t. v_t = beta2 v_(t-1) + (1-beta2) g_t^2. The update uses bias-corrected m divided by sqrt(bias-corrected v) plus epsilon.

A large beta2 makes the scale estimate adapt more slowly. This can smooth noise but respond sluggishly to changing gradient statistics. Epsilon prevents an unstable division and can matter in low-precision or very small-gradient regimes. Defaults are starting points, not proof of optimality.

CHECK: Optimizer state belongs to the training trajectory. Resuming only weights while resetting moments changes the experiment, even if the learning-rate value is identical. Record whether a run is an exact continuation, a warm start or a new optimizer phase.

Gradient norm is useful but incomplete

The global gradient norm summarizes the magnitude of all parameter gradients before an update. A sudden rise can reveal instability or an unusual batch. It does not by itself say whether the update is harmful, because adaptive rescaling, parameter scales and loss normalization all matter.

Gradient clipping replaces gradients by a scaled version when their norm exceeds a threshold. For global norm clipping, the common factor is approximately min(1, threshold / norm), with numerical safeguards. Clipping can contain occasional spikes, but constant heavy clipping can indicate an overly aggressive learning rate or another problem.

Log the pre-clipping norm and the fraction of updates clipped. A post-clipping norm alone hides the severity of the raw gradients. With loss scaling, unscale gradients before clipping. With accumulation, clip after the intended accumulated gradient is assembled.

DECISION: Use clipping as a documented stability mechanism, not as permission to ignore exploding activations, invalid labels or non-finite values. If every step hits the limit, investigate why.

Exercise: interpret two training traces

Run A has smoothly decreasing validation loss, occasional high gradient norms and clipping on two percent of updates. Run B clips on nearly every update, has erratic loss and sometimes produces non-finite gradients. Both use the same threshold.

Run A may be functioning normally; inspect whether spikes correspond to long or difficult batches. Run B needs investigation. Check the actual learning rate, loss normalization, precision handling and problematic batches. A lower peak rate is a useful controlled test after correctness checks.

ANSWER: Increasing the clipping threshold for B is not an obvious fix. It permits larger raw gradients to influence updates and can worsen instability. Lowering the threshold may reduce symptoms but still leave the underlying issue unresolved.

FOLLOW-UP: Compare pre-clipping norms under a fixed data order and record skipped updates if mixed-precision scaling is used. A nominal update count that includes skipped optimizer steps can misrepresent both training progress and scheduler state.

Choose a range, not a sacred constant

Learning rate is often one of the highest-value hyperparameters to investigate early. Too small can waste the available training budget; too large can cause unstable or ineffective updates. The right range depends on optimizer, parameterization, model size, batch, initialization and whether you train from scratch or fine-tune.

Begin from a coherent reference recipe close to your task, then use a small logarithmic sweep. For a small from-scratch AdamW language-model pilot, values such as 1e-4, 3e-4 and 1e-3 are illustrative search points, not guaranteed safe or optimal settings. Fine-tuning a pretrained model often calls for a different, frequently smaller range.

Hold the data order, training horizon, batch and schedule shape fixed within the pilot. Use the same warmup policy, expressed in meaningful updates or tokens. Evaluate validation loss rather than picking only the fastest training-loss drop.

DECISION: Select a rate that gives reliable progress and some stability margin. The largest rate that survives a short run can still fail later or generalize poorly.

Why logarithmic search is natural

Learning rates commonly span orders of magnitude. Sampling uniformly between 0.00001 and 0.01 puts most samples near the large end on a multiplicative scale. Log-uniform sampling gives comparable attention to each decade.

FORMULA: sample u uniformly between log(eta_min) and log(eta_max), then set eta = exp(u). A simple hand-built grid can multiply each value by about three.

Suppose 1e-4 is stable but slow, 3e-4 gives good validation progress and 1e-3 diverges. Refine around the useful region, perhaps testing 2e-4, 4e-4 and 6e-4 if budget permits. Do not assume the optimum is exactly halfway in ordinary arithmetic; use the scale suggested by the original search.

Keep the failed trial. Its configuration and failure time help bound the useful region. If all rates fail at the same batch, suspect the data or implementation rather than concluding that the entire interval is too large.

EXERCISE: The geometric midpoint of 1e-4 and 1e-3 is about 3.16e-4, not 5.5e-4.

A learning-rate range test is only a screen

A range test increases learning rate over a short run and observes how loss behaves. It can identify obviously ineffective and unstable regions cheaply. However, the weights change throughout the test, so each rate is not evaluated from the same initial state. Smoothing can also hide short failures.

Use the result to propose candidates, then reset to a common initial checkpoint and run separate controlled pilots. Do not continue the final training run from a range test that has visited unstable rates unless you deliberately accept that altered history.

The divergence point depends on batch order, warmup, precision and run length. A short screen cannot certify long-run stability. If your model is small and compute is available, a few independent short trials can be easier to interpret than an aggressively swept range test.

DECISION: Pick below the visibly unstable region and confirm with validation. The exact safety margin is empirical. Avoid rules such as always divide the divergence rate by ten unless you understand them as rough heuristics rather than laws.

Distinguish slow learning from insufficient capacity

If training and validation losses both improve very slowly, the rate may be too small, but other explanations exist. The model might be too small, the batch may be poorly constructed, gradients may be tiny due to a bug or the task may simply be difficult. Inspect updates before changing architecture.

If raising the rate yields faster training and validation improvement without instability, optimization was at least part of the bottleneck. If training improves but validation worsens, the larger rate is not automatically better. If no plausible rate allows a tiny-batch overfit, return to implementation checks.

Compare trajectories against processed tokens and elapsed time, using the same objective. Training losses from different batch compositions can be noisy, so evaluate a fixed training subset periodically under evaluation mode when you need a cleaner comparison.

CHECK: Log the learning rate actually stored in every optimizer parameter group. A configuration file can say 3e-4 while a scheduler or resumed optimizer applies something entirely different.

Worked learning-rate selection

Three hypothetical pilots use the same initialization, data order, batch and token budget. At the end, 1e-4 has validation loss 3.82, 3e-4 has 3.61 and 1e-3 has become non-finite. The middle rate is the strongest current candidate, but these pilot values are not measured results from your project.

Before committing, inspect the 3e-4 curve for late instability and confirm that its validation advantage is not due to a different checkpoint rule. If budget permits, test a nearby value and repeat the leading candidate with another seed. A large gap may not need extensive repetition; a tiny gap deserves more caution.

ANSWER: Choose 3e-4 as a justified provisional peak rate, not as the permanently correct value. Retune modestly after major changes to width, batch, objective or fine-tuning regime.

WRITE IN YOUR LOG: Selected from a controlled pilot because it improved validation fastest among stable candidates; final-run stability remains to be monitored. This sentence is more scientifically accurate than saying the optimal learning rate is 3e-4.

Warmup controls the early transition

Warmup increases the learning rate from a small initial value toward a target peak over an initial period. Early optimizer statistics and activations can be poorly calibrated, and a sudden large update can be harmful. Warmup is a practical stabilizing device, not a guarantee or a universal requirement.

Specify its units. One thousand optimizer updates can represent very different numbers of tokens when batch size changes. A warmup fraction also depends on the planned run horizon. If you double the horizon and preserve the fraction, you double the warmup duration.

For a first pilot, a small warmup fraction such as a few percent can be an illustrative starting policy, but short runs and different architectures may need different choices. Compare actual curves and avoid spending most of a tiny run warming up without realizing it.

DECISION: Choose peak learning rate and warmup together. If a peak rate fails immediately without warmup but works with it, that does not prove it will remain stable through the full run.

Linear warmup plus cosine decay

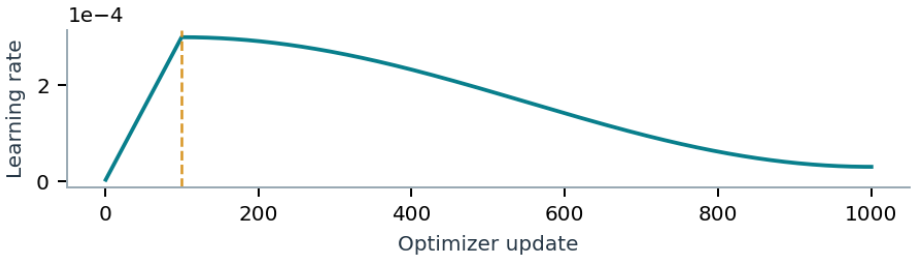
Let S be the planned number of optimizer updates and W the warmup updates. During warmup, increase learning rate linearly toward eta_peak. After warmup, a cosine schedule decays smoothly toward eta_min over the remaining horizon. Exact indexing conventions differ, so define yours explicitly.

FORMULA after warmup: eta(s) = eta_min + 0.5(eta_peak - eta_min)[1 + cos(pi (s-W)/(S-W))], for W <= s <= S.

Eta_min need not be zero. A nonzero floor preserves some update magnitude near the end, while a near-zero endpoint encourages a quieter finishing phase. The best endpoint depends on the training plan. A cosine schedule is horizon-dependent: changing S partway through changes the intended future trajectory.

CHECK: Plot the schedule before training and test the first, warmup-boundary and final rates. Off-by-one errors are common, especially when a scheduler is stepped per microbatch instead of per optimizer update.

PRACTICE: With eta_peak = 3e-4 and eta_min = 3e-5, the midpoint of the cosine decay has rate 1.65e-4.



Other schedules answer other needs

A constant rate is easy to interpret and can be useful in pilots. A constant rate followed by a deliberate decay phase separates exploration from finishing, but the decay plan still needs validation. Linear decay decreases at a fixed rate. Inverse-square-root schedules decay more slowly and have a different dependence on update count and warmup.

A plateau scheduler reduces learning rate when a monitored metric stops improving according to a threshold and patience rule. It fits some finite-data training workflows, but validation noise and evaluation frequency strongly affect when it triggers. It is not automatically superior to a predetermined schedule.

Restarts deliberately raise the rate again after decay. They change the training path and should not be confused with recovering from a crash. A restart can help in some settings, but it is not a generic remedy for all plateaus.

DECISION: Choose a simple schedule whose behavior you can explain. Compare schedules with similar tuning effort and a clearly defined horizon. Otherwise a carefully tuned cosine run can unfairly beat a poorly chosen constant-rate baseline.

Patience is measured in evaluations

If validation runs every 500 updates and patience is five evaluations, the scheduler waits roughly 2,500 updates without the required improvement. Changing evaluation frequency while keeping patience unchanged alters the effective training policy. The same problem applies to early stopping.

Define what counts as improvement: an absolute decrease, a relative decrease or a statistically meaningful change. A threshold smaller than validation noise can cause erratic decisions. A threshold too large can trigger decay while real progress continues.

Plateau logic should monitor a stable, relevant validation metric, not a single training minibatch. Save the scheduler's best-so-far metric, bad-evaluation count and cooldown state if you want a faithful resume.

EXERCISE: Evaluation every 2,000 updates with patience three can wait around 6,000 updates. If you switch to evaluation every 200 updates, the same patience can trigger after about 600. The integer three has not changed, but the experiment has.

CHECK: Record evaluation cadence alongside schedule settings in the resolved configuration.

Worked schedule comparison

Two hypothetical runs use peak learning rate 3e-4 and 20,000 updates. Run A stays constant; run B warms up for 500 updates and decays toward 3e-5. B finishes with better validation loss. This establishes that the complete B policy performed better in this experiment, not that cosine always wins.

To understand the result, inspect when the curves separated. Did warmup prevent an early setback? Did late decay improve the final checkpoint? Was A still improving at the end? A follow-up constant-then-decay run can separate some of these explanations if the question matters enough to justify the cost.

ANSWER: Use the schedule supported by your current budget and evidence, while keeping the claim narrow. The final metric alone does not explain which part of the schedule caused the improvement.

NEXT ACTION: Save a schedule plot with the experiment record. A short visual check often catches a warmup duration, floor or update-count mistake before it wastes a long run.

A plateau is a symptom, not an instruction

Do not reduce learning rate merely because one evaluation is worse. Stochastic optimization and finite validation sets create fluctuations. First determine whether the trend is genuinely flat over a meaningful interval and whether the current schedule already plans a decay.

Ask what is limiting progress. If training loss still falls while validation rises, overfitting is plausible and the best earlier checkpoint may be preferable. If both are flat, optimization, capacity, data quality or an exhausted schedule could be involved. If loss is unstable, investigate numerical and implementation issues as well as rate.

A rate reduction is a testable intervention: from a saved checkpoint, continue one branch unchanged and another at a lower rate for the same additional budget. This controlled continuation is more informative than changing the rate repeatedly in one run and attributing every later improvement to the latest change.

DECISION: Switch for a stated reason and keep the pre-switch checkpoint. You need a comparison path and a recovery point, not just optimism.

A safe manual reduction protocol

First save weights, optimizer state, scheduler state, update count, processed tokens and random states. Record the exact current rate in each parameter group. Decide whether the new phase replaces the previous schedule or modifies it. A scheduler can overwrite a manual rate assignment on its next step.

For an illustrative intervention, reducing by a factor such as two or three can test whether smaller updates help. The factor is not universal. Continue long enough to observe a trend, but define the budget before seeing the outcome. Compare with an unchanged continuation when affordable.

Usually retain Adam moments for a simple rate reduction. Resetting moments at the same time introduces another change. There are situations where restarting an optimizer is intentional, but label that as a distinct experiment.

CHECK: Log the applied rate immediately after the switch and after the next scheduler action. Verify that the new value persists as intended. A successful assignment in code does not prove the schedule uses it for subsequent optimizer updates.

Extending a finished cosine run

Suppose a cosine schedule has reached a very small endpoint and you decide to train longer. Simply adding more updates while leaving the endpoint rate unchanged may produce little learning. Recomputing the original cosine with a larger total horizon can abruptly raise the current rate. Both choices alter the training policy.

Define a new continuation phase explicitly. You might hold a modest low rate, use a new short warmup to a chosen continuation peak, or use a fresh decay segment. The right choice depends on the checkpoint, data and objective; none is an exact continuation of the original finished schedule.

Preserve the original checkpoint and compare continuation candidates if budget allows. A higher restart rate can temporarily worsen loss or erase useful fine adjustment. Monitor both training and validation after the transition.

DECISION: Treat horizon extension as a new experiment phase. Record its start token count and complete schedule. Do not silently redraw history by pretending the longer schedule was used from the beginning.

Changing batch or optimizer mid-run

Changing batch size changes gradient noise and the number of updates per processed token. Automatic linear scaling of learning rate is not a universal law, especially for adaptive optimizers. A modest retuning pilot is safer than assuming a factor based only on the batch ratio.

Changing optimizer is an even larger intervention. Adam moments do not map directly to SGD momentum in a uniquely correct way. The numerical learning rate should be reselected for the new update rule. If you switch because the current run is failing, first verify that a simpler rate or schedule adjustment is not sufficient.

When resuming after a crash, restore the original optimizer and schedule if the goal is continuation. When changing task or data distribution, a deliberate new phase may be appropriate. These are different purposes and should not share ambiguous labels such as resume-final.

CHECK: Update both step-based and token-based counters. If the batch changes, a schedule defined by tokens should continue from the actual processed-token count, while an update-based schedule has different semantics.

Worked continuation experiment

At update 12,000, validation loss has fluctuated around 3.20 for several evaluations. You save checkpoint C. Branch A continues at 2e-4; branch B continues at 7e-5. Both retain optimizer moments, use the same next data stream and receive 2,000 additional updates.

Suppose A ends at 3.19 and B at 3.13, with B showing a sustained improvement. This supports using the reduced-rate phase for this checkpoint and budget. It does not prove 7e-5 is globally optimal. A repeat or a longer continuation can test whether the gain persists.

If B initially improves but then stalls above A's eventual result, the short-budget ranking was misleading. Record the full trajectories. Do not choose a branch using a different checkpoint-selection rule for each.

ANSWER: Switch based on a controlled continuation when feasible. If compute permits only one branch, make the decision from the trend, preserve the old checkpoint and explicitly acknowledge that the causal comparison is weaker.

Count tokens, not just sequences

Sequence batch size can be misleading when examples have different lengths. For language modeling, valid target tokens per optimizer update are often the more informative quantity. Distinguish microbatch size, accumulation steps, number of devices and effective global batch.

For equal full-length sequences, global tokens per update = sequences per device times context length times accumulation steps times number of devices. Padding, loss masks and variable lengths require counting actual valid targets rather than relying on this product.

EXAMPLE: Two sequences per device, T = 512, eight accumulation steps and one device give 8,192 token positions per update if all targets are valid. Four devices give 32,768. This says nothing about how many unique corpus tokens have been seen overall.

DECISION: Choose a microbatch that fits efficiently, then use accumulation if you need a larger effective batch. Log both. A configuration that says batch_size = 2 without accumulation and device count is incomplete.

Larger batches change the optimization process

A larger batch generally reduces the noise of the gradient estimate, though examples are not always independent. It also gives fewer optimizer updates for a fixed number of processed tokens. Those changes can affect convergence and the useful learning-rate range.

Small batches can make noisy but frequent updates. Very large batches can waste compute if additional averaging brings little benefit while reducing the number of updates. There is no universal best batch size independent of model, data and budget.

Under an equal-token comparison, doubling batch halves the update count. Under an equal-update comparison, doubling batch doubles token exposure. State which one you are testing. If the schedule is expressed in updates, changing batch also changes how many tokens are spent at each learning-rate phase.

CHECK: Plot loss against tokens as well as updates. A model that improves faster per update may still be worse per token or per hour.

EXERCISE: At 8,192 tokens per update, 10,000 updates process 81,920,000 tokens, before any distinction between repeated and unique data.

Accumulation is approximately a larger batch

With equal-sized microbatches and a loss averaged per token, dividing each microbatch loss by the number of accumulation steps and summing gradients reproduces the large-batch average in ideal arithmetic. Dropout randomness, floating-point order and other implementation details can prevent bitwise identity.

Variable valid-token counts need more care. If one microbatch has 100 valid tokens and another has 900, averaging their mean losses equally gives the smaller one too much weight. Accumulate summed token losses and normalize by the total valid-token count for the effective batch, or use an equivalent weighted method.

Zero gradients at the start of an effective batch, not between its microbatches. Apply clipping after accumulation and after unscaling if using a loss scaler. Step the optimizer once, then update the schedule according to actual optimizer updates or your chosen token policy.

DECISION: Verify a tiny accumulation example against a direct large batch before trusting a long training run. Handle the final partial accumulation group explicitly; do not silently divide it by the wrong count.

Find an efficient batch on your hardware

Measure tokens per second across a few microbatch sizes while keeping context and model fixed. Very small microbatches may underutilize the device; larger ones can improve efficiency until memory or another bottleneck dominates. The fastest feasible microbatch is not always the best effective batch for optimization.

Accumulation can increase effective batch without increasing the largest activation allocation as much as a direct larger batch. It does not reduce model weights or optimizer-state memory, and it adds sequential microbatch work. Distributed synchronization policies can also affect its speed.

If memory is tight, inspect padding, sequence length, activation checkpointing and attention implementation before shrinking model capacity automatically. Each intervention changes a different part of the cost.

DECISION: Select microbatch size for efficient execution, then choose effective batch and learning rate jointly for training behavior. Keep a memory margin for longer examples and evaluation rather than filling the device to the last measured byte.

Worked batch-change decision

Your stable baseline uses 4,096 tokens per update at peak rate 3e-4. A faster implementation makes 16,384 tokens per update attractive. You propose multiplying the rate by four and keeping the same update count.

That changes both update magnitude and total token exposure by large factors. It is not a controlled test of batch size. A better initial comparison preserves the total token budget, adjusts schedule units deliberately and tests a small range of rates for the larger batch.

ANSWER: There is no guaranteed factor-four learning-rate rule for this AdamW Transformer. Start from a cautious range informed by the baseline and measure. If the larger batch is faster but slightly less token-efficient, the final decision depends on whether your constraint is time or data.

FOLLOW-UP: Record the number of optimizer updates after the change. A warmup of 500 updates now covers four times as many tokens, so leaving that integer unchanged can materially alter the training policy.

Weight decay is separate from gradient fitting

AdamW applies decoupled weight decay rather than simply adding an L2 penalty to the gradient before Adam's adaptive rescaling. A simplified update contains a shrinkage term proportional to learning rate times weight-decay coefficient. This distinction matters because adaptive preconditioning changes the effect of an ordinary L2 gradient term.

FORMULA, schematically: theta_next = (1 - eta lambda) theta - eta times adaptive_gradient_update.

The cumulative shrinkage depends on the learning-rate trajectory and number of updates, not only on lambda. If you change batch size at fixed tokens, you change update count; if you change the schedule, you change the sum of rates. The same nominal decay coefficient can therefore have a different total effect.

DECISION: Treat decay as a regularization and optimization choice tied to the training recipe. Do not compare two decay values while silently changing the horizon or parameter groups and assume that only one factor changed.

Decide which parameters receive decay

Many Transformer recipes exclude biases and normalization scales from weight decay while applying it to matrix weights. Embeddings and tied output matrices require an explicit policy; conventions differ. The correct grouping depends on the intended recipe, not solely on a parameter's name.

A practical grouping rule often distinguishes matrix-like tensors from one-dimensional parameters, but it can fail for unusual modules. Inspect the groups and ensure every trainable parameter appears exactly once. Shared parameters must not be duplicated across groups with conflicting settings.

EXERCISE: If a tied embedding/output matrix is listed twice by a custom traversal, what can go wrong? It may be updated twice or rejected by the optimizer, and its intended decay policy becomes ambiguous. Use the framework's parameter handling carefully and verify uniqueness.

CHECK: Log group names, parameter counts, learning rates and decay values. A model with correct code but an optimizer that omits half its tensors can look like an architecture that refuses to learn.

Beta values control memory of gradients

Beta1 controls the averaging timescale of the first moment; beta2 controls that of squared gradients. A rough characteristic timescale is about 1/(1-beta) updates. This is an intuition for exponential averaging, not an exact statement that all older information disappears after that many updates.

For beta2 = 0.999, the rough scale is 1,000 updates. For 0.99 it is 100. If the gradient distribution changes quickly, a slower scale estimate may lag; a faster one may be noisier. Different Transformer recipes use different choices, so copying one beta2 without its batch and schedule context can mislead.

Keep beta values fixed during the first learning-rate sweep unless there is a specific stability reason to investigate them. Too many simultaneous optimizer variables create a search space you cannot afford to interpret.

DECISION: Start with a conventional AdamW recipe, then investigate moments only after checking learning rate, loss normalization, data and numerical stability. A beta change is not a substitute for diagnosing the failure.

Epsilon and numerical precision

Adam's epsilon prevents division by a very small denominator. Its effect depends on gradient scales and arithmetic precision. Changing epsilon can alter updates when the estimated second moment is small. It is usually a lower-priority tuning dimension than learning rate, but it can matter in numerical failures.

Know where optimizer states are stored and which operations run in full precision. Low-precision forward computation does not imply every optimizer operation uses that same dtype. FP16 and BF16 have different exponent ranges and precision characteristics, so their failure modes differ.

If NaNs appear, locate the first non-finite tensor rather than changing epsilon immediately. Check inputs, logits, loss, gradients and optimizer states in order. A divide-by-zero in your data preprocessing is not an Adam epsilon problem.

CHECK: Record optimizer implementation, state dtypes and whether a fused implementation is used. These details can affect memory, performance and exact reproducibility even when the high-level hyperparameters match.

Worked decay comparison

Two runs use lambda = 0.1. Run A trains for 10,000 updates at a constant rate of 1e-4; run B uses 20,000 updates at the same rate. Ignoring the gradient term, A's repeated shrinkage is approximately exp(-0.1), while B's is approximately exp(-0.2), using the small-step exponential approximation.

The same decay coefficient has produced different cumulative shrinkage. Real training includes gradients, so this does not predict final weight norms exactly. It does show why quoting lambda alone is incomplete.

ANSWER: When comparing decay settings, keep schedule and update budget controlled or explicitly analyze their change. Search a small range appropriate to the regime; zero decay is a useful baseline when you need to understand whether decay helps.

FOLLOW-UP: In fine-tuning, overly strong decay can damage pretrained weights or task adaptation. In from-scratch training, insufficient regularization may matter differently. Do not transfer a value merely because both experiments use AdamW.

Overfitting is a relationship between curves

Overfitting is suggested when training performance improves while held-out performance stops improving or worsens, under comparable measurements. It is not diagnosed simply because training loss is lower than validation loss. A gap is expected in many settings, and dropout or different objectives can distort comparisons.

Evaluate a fixed training subset and validation set in evaluation mode using the same unsmoothed metric when you want to interpret the gap. Examine whether the validation distribution matches the intended task. A large domain shift can produce a persistent gap without ordinary memorization being the main explanation.

Regularization includes explicit mechanisms such as dropout and weight decay, but also data quality, augmentation where valid, capacity choice and checkpoint selection. Early stopping limits training duration based on validation evidence. None of these guarantees a universal best model.

DECISION: Diagnose the cause before choosing a regularizer. Increasing dropout on a model that is already underfitting can make learning worse.

Dropout changes the training computation

Dropout randomly removes or scales components during training and is disabled during ordinary evaluation. Its location matters: attention probabilities, residual branches and feed-forward activations are different places to apply it. A single dropout number may control several locations in a library implementation.

For a small finite-data model, values such as 0, 0.1 and 0.2 can be illustrative candidates. Large-scale pretraining recipes sometimes use little or no dropout; that does not imply zero is always best for a small repeated corpus. Context, data scale and training duration matter.

If training loss is measured with dropout active and validation loss without it, validation can occasionally be lower without any bug. That observation is not proof of data leakage or miraculous generalization. Compare under consistent modes before drawing conclusions.

CHECK: Ensure model.train() and model.eval() are set correctly, and restore training mode after validation. Accidentally training in evaluation mode can silently remove intended regularization.

Label smoothing and objective changes

Label smoothing replaces a completely concentrated target distribution with one that assigns some mass elsewhere. It can discourage overconfident predictions and is used in some classification and sequence-to-sequence settings. Its effect depends on the task, vocabulary and evaluation objective.

A smoothed cross-entropy is not the same as ordinary negative log-likelihood of the true token. If you exponentiate the smoothed objective and label it ordinary perplexity, the interpretation is wrong. Use the unsmoothed true-target loss for conventional perplexity reporting.

Do not add label smoothing automatically to every language-model pretraining task. Treat it as an objective choice requiring a reason and validation. For translation, compare generated quality as well as token loss, with decoding settings held fixed.

DECISION: When the objective changes, state the change prominently. A lower numerical training loss under a different objective does not establish a better model. The comparison must use a common evaluation metric.

Epochs are not a universal training budget

An epoch is one pass through the chosen dataset under its sampling rules. For a fixed small corpus, extra epochs repeat the same information and can increase memorization. For streaming mixtures or sampling with replacement, an epoch may be arbitrary or poorly defined.

Track processed tokens, unique data where measurable and optimizer updates. If the tokenizer changes, token count changes meaning. If you deduplicate the corpus, an epoch becomes smaller but may contain a higher fraction of useful information. These distinctions matter when transferring a schedule from another project.

Select the best checkpoint using a predetermined validation rule. Early stopping should have patience and a meaningful improvement threshold; it should not react to every small fluctuation. Keep a final test set untouched until the configuration and selection rule are fixed.

EXERCISE: A 10M-token corpus trained for ten passes has 100M token exposures, not 100M unique tokens. Scaling arguments based on fresh data should not be applied as though those quantities were identical.

Worked regularization choice

A hypothetical small language model has training loss 2.4 and validation loss 3.3 at its final checkpoint. Earlier, validation reached 3.0. Increasing model size lowers final training loss to 2.0 but leaves validation at 3.4. The data and metric checks are clean.

The best earlier checkpoint is already preferable under the validation objective. Next experiments could shorten training, add appropriate dropout, adjust decay or improve data coverage. Test these separately or in a small planned design. A larger model is not the most evidence-supported next move.

ANSWER: Keep the checkpoint with the best validated performance and investigate generalization. Do not report the final checkpoint merely because it was saved last. Also do not claim that regularization must close every gap: some gap reflects unavoidable distribution differences or limited data.

FOLLOW-UP: If additional clean data improves validation more than regularization does, the project has learned a valuable resource-allocation lesson. Hyperparameter expertise includes knowing when the best intervention is better data.

Precision is an execution choice with consequences

FP32, FP16 and BF16 trade storage, arithmetic precision and numerical range differently. Mixed-precision training uses lower precision for selected operations while preserving appropriate higher-precision computation or state elsewhere. The exact policy depends on hardware and software.

FP16 has a narrower exponent range than BF16 and often uses dynamic loss scaling during training. BF16 has a wider exponent range but fewer fraction bits than FP32. Neither format guarantees numerical stability, and support or speed depends on the device. Do not assume the same mixed-precision recipe works everywhere.

For a custom model, first establish a correct small FP32 baseline when feasible. Then enable mixed precision and compare short loss traces, finite-gradient behavior and throughput. Exact trajectories can differ due to rounding; large systematic discrepancies deserve investigation.

DECISION: Treat precision changes as controlled engineering changes. Record compute dtype, parameter dtype, optimizer-state dtype and loss-scaling behavior, rather than writing only mixed_precision = true.

Understand loss scaling and skipped updates

Loss scaling multiplies the loss before backward so small gradients are less likely to underflow in limited precision. The scaler later unscales gradients before the optimizer update. If non-finite gradients are detected, an update may be skipped and the scale adjusted.

Clip only after unscaling. Otherwise the clipping threshold applies to artificially scaled gradients and can severely distort updates. Log skipped updates when they occur. A run that appears to perform 10,000 iterations may have fewer actual optimizer updates.

Schedulers must follow the intended progress measure. A scheduler that advances on every attempted step, even when the optimizer skips updates, can move through the learning-rate policy without the expected learning. Handling this correctly can depend on the framework API, so verify your installed implementation.

CHECK: Inspect gradients and scaler behavior during a short pilot. A few initial adjustments may be normal; persistent non-finite gradients require diagnosis. Lowering learning rate is one possible test, not a substitute for finding invalid operations.

Activation checkpointing trades time for memory

During ordinary backpropagation, intermediate activations are stored so gradients can be computed later. Activation checkpointing saves fewer intermediates and recomputes portions of the forward pass during backward. This can enable a larger model, batch or context at the cost of additional computation.

Checkpointing does not eliminate parameter, gradient or optimizer-state memory. If those persistent allocations dominate, its benefit may be limited. Different checkpoint boundaries create different time-memory tradeoffs, and stochastic operations require correct handling of random state if equivalent behavior is expected.

Measure the result rather than assuming a fixed memory-saving percentage. Kernel choices, sequence length and architecture change the balance. A checkpointed configuration that fits but is much slower may be inferior under a strict training-time budget.

DECISION: Use checkpointing when the memory it saves enables a worthwhile experiment. Compare quality per hour after the change, not just the maximum model size you can instantiate.

Profile before optimizing the wrong thing

A slow run may be limited by the data loader, padding, CPU preprocessing, device transfers, small matrix shapes, attention or synchronization. Changing model hyperparameters to compensate for an input pipeline bottleneck can sacrifice quality without solving the main issue.

Measure sustained valid tokens per second after warmup, with realistic batches. Include evaluation overhead separately. Device operations can be asynchronous, so naive host timing around a kernel launch may not measure completed work. Use appropriate synchronization or profiling tools for the environment.

Record peak memory over a real training step including backward and optimizer update. Test long examples if the dataset is variable-length. A single short batch is not a reliable upper bound.

WORKSHEET: For each candidate, record model parameters, context, valid tokens/update, precision, checkpointing, measured throughput, peak memory and evaluation time. This turns vague speed impressions into a resource model you can use to plan experiments.

Worked out-of-memory diagnosis

A model fits its first forward pass but fails during the first optimizer update. This pattern suggests that optimizer states or other update-time allocations may be the missing memory cost. Reducing context might help activations but may not address the dominant new allocation.

First measure the stages separately: model construction, forward, backward and first update. If persistent optimizer memory dominates, consider a smaller model or a different supported optimizer/state strategy, understanding the changed training recipe. If activations dominate, consider a smaller microbatch, checkpointing or a more memory-efficient attention path.

ANSWER: Diagnose the allocation before selecting the remedy. Gradient accumulation can preserve effective batch when you reduce microbatch, but it does not shrink optimizer states. A larger accumulation count is not a universal memory fix.

FOLLOW-UP: Keep a safety margin. Real batches, validation, generation or checkpoint serialization can create additional peaks. A configuration that barely survives one synthetic step is not yet a dependable training setup.

Validation is part of the experiment design

Define validation data, metrics, checkpoint selection and evaluation frequency before comparing configurations. If you change those rules after seeing results, the comparison becomes harder to interpret. The purpose of validation is to guide choices while approximating the target deployment distribution.

Use evaluation mode and disable gradient tracking for ordinary metric computation. Aggregate token losses correctly across batches and devices. Keep the same tokenizer, masking policy and text preprocessing when comparing perplexity. A metric pipeline bug can make a bad model look excellent.

Separate the primary metric from supporting diagnostics. For a language model, validation negative log-likelihood may be primary, with generation inspection and task-specific tests supporting it. For translation, sequence-level quality and human review may be necessary alongside loss.

DECISION: Write the evaluation contract in the experiment configuration. Include the exact split version, valid-token denominator, decoding policy where relevant and checkpoint selection rule. Reproducibility begins before the first run.

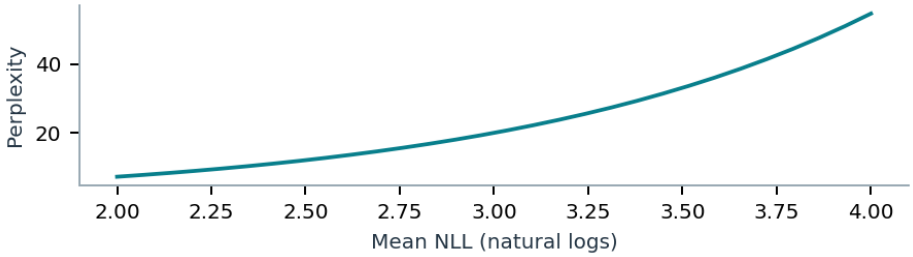
Perplexity is exponential, not mysterious

Perplexity is exp(mean negative log-likelihood) when natural logarithms are used. It is a transformation of the same token prediction loss, not an independent metric. Lower is better for the same evaluation setup, but the numeric scale depends on tokenizer and dataset.

If loss falls from 3.0 to 2.9, perplexity changes from about 20.09 to 18.17. The relative perplexity factor is exp(-0.1), about 0.905. A small-looking loss change can therefore correspond to a noticeable multiplicative change in perplexity.

Do not average per-batch perplexities to obtain dataset perplexity. First aggregate the token negative log-likelihood correctly, then exponentiate. Exponentiation is nonlinear, so averaging exponentiated batch losses gives a different result.

EXERCISE: Two equal-token batches have losses 1 and 3. Dataset perplexity is exp(2), about 7.39. The average of exp(1) and exp(3) is about 11.40 and is not the desired dataset value.



Seeds and validation samples create different noise

Different training seeds change initialization, dropout and data order. A finite validation set also introduces uncertainty about deployment performance. Repeating training seeds addresses one source of variation; evaluating more independent validation data addresses another. They are not interchangeable.

For close finalists, repeat a few seeds when affordable and report individual results or a mean with spread. With very few runs, avoid overstating precision. A tiny difference can be less important than a clear throughput or memory advantage.

If you use bootstrap intervals for language data, resample at an appropriate independent unit such as documents, rather than pretending every adjacent token is independent. For paired model comparisons, preserve the pairing of evaluation examples. Statistical details matter when differences are small.

DECISION: Do not demand expensive repetition for every obviously failed pilot. Reserve it for decisions where uncertainty could change the conclusion. A professional uses uncertainty estimates to guide resource allocation, not to decorate every table with unjustified decimals.

Multiple comparisons can fool you

If you try many configurations and select the lowest validation score, some of the apparent win can be luck. Repeatedly inspecting the same validation set also adapts the development process to that set. This is why a final held-out test and honest reporting of search effort matter.

A baseline tuned with one trial and a proposed method tuned with fifty trials are not receiving equal optimization effort. If the proposed method wins slightly, some advantage may come from search budget rather than the underlying idea. Report the search protocol.

Keep an untouched final test for the chosen configuration. If you use its results to select another model, it has become part of development; a new independent evaluation is needed for an unbiased final estimate.

CHECK: Record how many configurations were attempted, including failures and abandoned runs. The best score without the search history is an incomplete result.

Worked finalist selection

Candidate A has validation losses 3.12, 3.15 and 3.13 across three seeds. Candidate B has 3.11, 3.18 and 3.14. B is also twenty percent slower. These hypothetical results do not establish a reliable quality advantage for B, despite its best individual run being lower.

A reasonable choice is A, especially under a time constraint. Report the individual values and the cost difference. If a specific downstream metric strongly favors B, evaluate that tradeoff explicitly rather than choosing by the single best loss.

ANSWER: Select using the predetermined metric and a consistent checkpoint policy. Do not compare A's mean against B's best seed. Do not claim statistical certainty from three runs without an appropriate analysis and assumptions.

FOLLOW-UP: Evaluate the selected configuration on the final test once the decision is frozen. Include limitations such as small seed count, finite data and the scope of the task. Good reporting preserves uncertainty without making the result useless.

First establish what the curves measure

Before interpreting a curve, identify the x-axis, loss definition, averaging window, model mode and data source. Training minibatch loss with dropout and smoothed targets is not directly comparable to evaluation-mode unsmoothed validation loss. A moving average can hide both spikes and recent improvements.

Plot raw or lightly aggregated training values alongside a readable trend, and mark validation points separately. Add the learning-rate trajectory and important events such as data changes, checkpoint resumes or batch-size changes. Many apparently mysterious curve changes align with a known intervention.

Use tokens or wall time as additional x-axes when update counts differ between experiments. A model that reaches a score in fewer updates can still consume more tokens or more time.

DECISION: Diagnose from multiple signals: training loss, validation loss, learning rate, gradient norms, clipping frequency, throughput and representative predictions. No single curve uniquely identifies every failure mode.

Both losses stay high

If both training and validation losses remain high, check correctness first. Verify shifted targets, masks, finite inputs, nonzero gradients, optimizer parameter coverage and actual updates. Run the tiny-batch overfit test. These checks are cheaper and more decisive than immediately increasing capacity.

If correctness is established, test learning-rate range and training duration. A rate that is too small can look like undercapacity. An unstable large rate can also make little net progress. A short run can leave a perfectly capable model undertrained.

Only after those checks should capacity be a leading explanation. Compare a modestly larger width or depth under a fair budget and a small optimizer retune. Also examine whether the task or data has irreducible ambiguity; some loss cannot be eliminated by architecture.

NEXT TEST: Ask which intervention separates the hypotheses most cheaply. A tiny-batch memorization test separates implementation failure from some capacity questions more effectively than a full twelve-hour sweep.

Training improves, validation worsens

This pattern is consistent with overfitting, but validate the measurement and split first. Check that validation preprocessing has not changed and that evaluation uses the intended model mode. Confirm that a corrupted or unusually small validation subset is not producing misleading fluctuations.

If the pattern is sustained, retain the best earlier checkpoint. Investigate training duration, data quality, distribution coverage, capacity and regularization. More clean training data may be more useful than stronger dropout. A smaller model can be a good choice when the data budget is limited.

Do not reduce learning rate automatically and assume it will cure overfitting. Smaller updates can continue fitting the training data even more closely. A controlled continuation may help, but checkpoint selection and generalization interventions are central.

CHECK: Compare performance by category. If validation worsens only on one domain while improving elsewhere, the problem may be a data-mixture tradeoff rather than a uniform failure of generalization.

Spikes, NaNs and sudden discontinuities

A single loss spike can arise from a difficult batch, long sequences, unusual tokens or noisy optimization. Repeated escalating spikes suggest a more serious issue. Non-finite values require locating the first failing stage: data, forward activations, logits, loss, gradients or optimizer state.

Save or identify the offending batch when possible. Replaying it from a safe checkpoint can distinguish data-triggered failure from a general training instability. Check loss normalization when valid-token counts vary; a sum used where a mean was expected can change gradient scale dramatically.

A discontinuity exactly at resume may indicate missing optimizer state, a scheduler reset, changed data order or altered precision. A discontinuity after evaluation may indicate that training mode was not restored. These are concrete hypotheses you can test.

DECISION: Stop a clearly non-finite run and preserve diagnostics. Continuing to spend compute on corrupted weights rarely produces interpretable evidence. Recover from the last known finite checkpoint only after deciding what caused the failure.

Worked diagnostic tree in words

Suppose loss is flat at approximately log(V), the value associated with uniform token probabilities, and generated text is random. First inspect logits and parameter changes. If parameters do not change, check requires_grad, optimizer groups, backward, step and the actual learning rate.

If parameters change but the tiny batch cannot be memorized, inspect label alignment and causal masking. If the tiny batch can be memorized but full training barely improves, inspect the learning-rate range, data diversity and training budget. If training improves but held-out loss does not, investigate generalization and split quality.

ANSWER: The same visible symptom can arise from several causes. Follow tests that eliminate explanations instead of attaching one diagnosis to one curve shape. Uniform-loss intuition is useful, but real initialization, target frequencies and output biases can make the initial loss differ from log(V).

PRACTICE: Write a three-step diagnostic plan for each of these symptoms: only long batches fail; loss jumps after resume; validation improves while generated repetition worsens. Each plan should include an alternative explanation.

Search is an experiment, not a lottery

Random search can be effective because only some dimensions may strongly affect results, and a full grid spends many trials repeating unimportant combinations. Bayesian optimization uses observed results to propose further candidates. Successive-halving methods allocate more budget to promising trials. Each method has assumptions and overhead.

For a small student project, a structured manual search often works well: validate correctness, bracket learning rate, compare a few capacities, refine regularization and confirm finalists. The structure matters more than using an impressive optimization library.

Define the search space, budget and selection rule before launching. Constrain invalid combinations such as widths not divisible by head count in an implementation that requires that relationship. Avoid ranges so broad that most trials are impossible or obviously unstable.

DECISION: Use search machinery to answer a clear question. It cannot rescue a leaked validation split, incorrect loss or unfair compute comparison.

Use distributions suited to the variable

Learning rate and sometimes weight decay are naturally searched on multiplicative scales. Depth and head count are discrete choices. Width should respect implementation and hardware constraints. Dropout lies in a bounded interval and should include sensible low values for the regime.

Conditional parameters require conditional spaces. A plain feed-forward network and a gated network use different parameter-count formulas. A plateau schedule needs patience and threshold; a fixed cosine schedule needs a horizon and floor. Sampling every field independently can create incoherent configurations.

EXAMPLE SEARCH: First test three peak rates for one baseline. Then compare two widths using the leading rate region with modest retuning. Finally compare two regularization settings on the best architecture. This staged search will miss some interactions, but it is interpretable and affordable.

CHECK: Keep a record of the search effort spent on each candidate family. A family receiving more tuning opportunities has an advantage that should be acknowledged.

Early stopping of bad trials has risks

Short trials can identify divergence and gross inefficiency. They can also mis-rank models whose learning curves cross. A deeper model, a longer warmup or a different schedule may look worse early and better later. Successive-halving methods assume early performance is informative enough to allocate budget.

Use comparable progress units such as processed tokens, not simply epoch count when datasets or tokenization differ. Ensure a trial has passed enough of its warmup to make its score meaningful. Do not eliminate a candidate because its rate has not yet reached the intended operating range.

Reserve a small amount of budget to check whether early rankings persist for a few diverse candidates. This guards against a search process that always favors fast starters. However, if the real deployment requirement is a very short training budget, fast starting can itself be a legitimate objective.

DECISION: State whether you seek best short-budget performance or best final performance under a longer horizon. Early stopping should serve that objective rather than silently redefine it.

One-factor and factorial experiments

Changing one factor at a time is easy to interpret but can miss interactions. Learning rate and batch size interact; depth and initialization interact; context and token batch interact. A small factorial design can reveal whether the effect of one choice depends on another.

For example, test two learning rates at two batch sizes, producing four configurations. Compare the rate effect within each batch and the batch effect within each rate. If the ranking reverses, a single global statement about the best rate is inappropriate.

Do not expand to a huge factorial grid without a budget. Use interaction experiments where theory or diagnostics suggest they matter. For lower-priority choices, a simpler staged search can be sufficient.

EXERCISE: If low rate wins at small batch and high rate wins at large batch, what is the conclusion? The useful rate depends on batch in this experiment. It does not prove a universal linear scaling formula or identify the optimum outside the tested values.

Worked twelve-run plan

Suppose you can afford twelve medium-length pilot runs plus one final run. Spend three pilots bracketing learning rate on a validated baseline. Spend four on two architecture candidates with two plausible rates each. Spend two on a targeted regularization or schedule question. Use the remaining three to confirm close finalists or investigate a suspected interaction.

This is an illustrative allocation. If the first pilots reveal a bug, pause and repair it rather than mechanically using the remaining slots. If one candidate wins by a large margin, spend confirmation compute where it can still change a decision.

ANSWER: A search plan should adapt to evidence while keeping its changes documented. It should not become a sequence of unrecorded guesses. Record failed runs, consumed time and why each next trial was selected.

FINAL RULE: Choose the final configuration using the validation protocol, freeze it, then evaluate on the test set. If the final run has a new failure, report and investigate it instead of selecting a convenient test-winning checkpoint after the fact.

Build a baseline you can understand end to end

For a first serious language-model study, a small decoder is often more educational than an ambitious model you can barely train. Choose a clean text corpus, document-level split, fixed tokenizer and manageable context. Define what you want it to do beyond producing amusing samples.

An illustrative baseline could use V = 8,000, d = 256, L = 6, h = 4 or 8, a plain f = 1,024 network and tied embeddings. Context might begin at 256 or 512 if that fits the task. These are candidate values, not an endorsed optimum for an unseen dataset.

Compute the main count: embeddings contribute 2,048,000 and blocks contribute 4,718,592, totaling about 6.77M before small terms and positions. This is small enough to make repeated controlled experiments more plausible than with a much larger model.

DECISION: Start with a model whose failures you can afford to investigate. The first success criterion is a correct, reproducible learning pipeline, not a round parameter count.

Stage one: correctness and measurement

Inspect tokenized examples, shifts and masks. Run a forward/backward/update test with finite-value checks. Overfit a tiny fixed batch with diagnostic settings. Confirm that validation mode and loss aggregation work. Save and reload a checkpoint and verify that the stored counters and optimizer state are present.

Measure sustained throughput and peak memory for the actual context and microbatch. Estimate how many tokens fit your budget. Decide how often to evaluate based on both cost and the timescale of expected changes. Very frequent full validation can consume a surprising fraction of a short project.

Create a baseline configuration and a run log before beginning the search. Include tokenizer hash, split definition, seed, code revision, architecture, optimizer, schedule and precision. Store representative fixed prompts for generation inspection.

EXIT CRITERION: The model learns the tiny batch, training decreases sensibly on real data, evaluation is reproducible enough to compare and a resumed run uses the intended learning rate. Only then proceed to quality tuning.

Stage two: optimization before architecture

Run a small peak-learning-rate sweep with a simple schedule and fixed token budget. Use the pilot to reject unstable and clearly slow choices. If all rates perform poorly, return to the data and implementation checks instead of immediately adding blocks.

Next investigate whether the run is still improving when the budget ends. If so, training duration may be a better investment than architecture expansion. If training loss remains high after optimization is credible, compare one wider or deeper model with modest rate retuning.

Keep sampling settings fixed when inspecting outputs. Greedy output, temperature sampling and top-p sampling can look very different for the same checkpoint. Use validation loss as a stable primary measure and samples as supporting evidence.

DECISION: Your first architecture comparison should answer a specific capacity question. For example: does increasing width from 256 to 384 improve validation enough to justify its measured training and generation cost? That is more informative than changing six settings at once.

Stage three: a compact experiment report

Write a report containing the task, data split, tokenizer, model counts, training budget, search protocol and final results. Include loss-versus-token and loss-versus-time plots, plus peak memory and a few fixed-prompt outputs. Label all synthetic planning numbers separately from measured results.

Explain the most important failed experiment. A failed larger model can reveal undertraining; a failed high rate can bound the stable region; a failed dropout change can show that the model was already underfitting. These observations demonstrate understanding better than hiding every unsuccessful run.

State what remains unresolved. Perhaps two head counts are indistinguishable at the current seed count, or longer context has not been evaluated. That is a useful next-step statement, not a weakness to conceal.

CHECK: Could another student reproduce your baseline without asking for missing values? If not, the project report is not complete. The exact configuration and data preparation matter more than a polished description alone.

Worked small-model decision

Your 6.77M baseline trains stably. A 10M candidate improves validation by 0.08 loss at equal tokens but is thirty percent slower. Under equal wall time, the baseline reaches nearly the same loss because it sees more tokens. These hypothetical outcomes show a token-efficiency versus time-efficiency tradeoff.

If your constraint is a limited fixed dataset, the larger model may still be attractive, provided extra passes do not substitute for fresh information in a misleading way. If your constraint is a short GPU allocation, the baseline may be the better choice. Add inference cost if you plan to deploy it.

ANSWER: There is no single winner without the objective. Report both views and choose based on the project constraint. Do not call the larger model better merely because its equal-token score is lower.

NEXT EXPERIMENT: If both are close at the time budget, test whether a better schedule or cleaner data improves the baseline more cheaply than another capacity increase.

A target size is a constraint, not a reason

An approximately 80M-parameter model can be a useful engineering goal, but the number itself does not guarantee capability. Data quality, training tokens, optimizer and compute budget determine whether that capacity is used. A well-trained smaller model can outperform an undertrained larger one under a fixed project budget.

Use the target to construct feasible candidates, then ask whether you can train them meaningfully. Keep embeddings, output tying and feed-forward design explicit. A model described as 80M can have very different depth-width allocation from another with the same count.

For a conventional tied decoder, N approximately Vd + 12Ld squared gives a first estimate. This guide's worked candidate uses V = 32,000, d = 640 and L = 12. It is an educational candidate, not a reproduction of a named model or a validated recommendation for your hardware.

DECISION: Do the arithmetic and a throughput pilot before committing. If the budget is insufficient, reduce the target size rather than treating undertraining as a mysterious hyperparameter failure.

Count the worked candidate

Embeddings contribute 32,000 times 640 = 20,480,000 weights. Each standard block with f = 2,560 contributes approximately 12 times 640 squared = 4,915,200. Twelve blocks contribute 58,982,400, giving a main-weight estimate of 79,462,400.

A possible head partition is h = 10 with d_h = 64. Other valid partitions are possible; ten is chosen here because it matches this illustrative head dimension, not because ten heads have special status. Biases, normalizations and any learned positional table increase the actual total.

If you use an untied output projection, add roughly another 20.48M weights. If you change to a gated feed-forward network at the same intermediate width, the count also rises. Recount after every structural change.

EXERCISE: With learned positions at T = 1,024, add 655,360 position weights. With a non-parameterized rotary position mechanism, that particular learned table is absent, although other implementation details still need counting.

Scaling laws are guides, not commandment tables

Compute-optimal scaling studies examine how model size and training data should grow under specific compute assumptions and experimental regimes. They motivate balancing parameters and tokens rather than maximizing parameters alone. Their fitted constants and ratios are not universal laws for every small project, dataset or deployment goal.

A commonly discussed rough ratio from Chinchilla-style reasoning is around twenty training tokens per parameter in that study's regime. For 80M parameters, that arithmetic suggests about 1.6B tokens. Treat it as context for scale, not a requirement that exactly 1.6B tokens makes your model optimal.

Repeated exposure to a tiny corpus is not equivalent to the same amount of fresh diverse data. Inference-constrained training can also favor smaller models trained on more tokens than a training-compute-only optimum would suggest. Data quality and tokenizer choices complicate comparisons further.

DECISION: Use scaling ideas to ask whether your plan is severely unbalanced, then use measured pilots and actual task constraints. Do not turn a research trend into a precise guarantee for an unseen student dataset.

Turn the size into a feasible run plan

Instantiate the candidate and measure a complete training step. Find a microbatch and context that fit with margin. Choose effective token batch using accumulation if needed, then bracket peak learning rate with a few short pilots. Keep the main run's token budget and schedule explicit.

Estimate time using measured valid tokens per second. If you can process 10,000 tokens per second, 1.6B token exposures would take about 44.4 hours before extra overhead. That throughput is hypothetical; your machine may be far faster or slower. A planning calculation must not be presented as a benchmark.

Decide whether the available data supports the intended token exposure. If the corpus is small, monitor repeated-pass overfitting and consider a smaller model or more data. Save frequent recoverable checkpoints because a long run has more opportunities for interruption.

CHECK: Include validation and checkpoint time in the budget. A nominal 48-hour allocation can leave much less than 48 hours of optimizer work.

Worked go/no-go decision

Your pilot shows that the near-80M candidate needs about thirty hours for the minimum token exposure you consider meaningful, but you have eight reliable GPU-hours. A smaller candidate can receive several times as many tokens in that time and has already shown promising validation progress.

A sensible decision is to finish the smaller model and document the 80M design as a future experiment. Alternatively, use the large model for a narrowly scoped engineering demonstration, while clearly stating that it is undertrained and not a quality comparison winner.

ANSWER: Expertise includes declining a configuration that the budget cannot support. A parameter-count milestone is not worth obscuring the actual training limitation. You can still demonstrate architecture design, memory accounting and checkpoint reliability without claiming mature language quality.

NEXT ACTION: Revisit the larger candidate when you have better data, more compute or a task where its extra capacity shows a measured benefit in short pilots. Keep the decision tied to evidence rather than a resume-friendly round number.

Fine-tuning starts from a different place

From-scratch training learns a representation from random initialization. Fine-tuning begins with useful pretrained weights and adapts them to a new objective or distribution. The same architecture can therefore require a different learning-rate range, training duration and regularization strategy.

A large update can damage useful pretrained behavior. A tiny update may fail to adapt. Start from a recipe appropriate to the model family and task, then run a small controlled rate sweep. Do not copy a from-scratch peak rate merely because the optimizer is still AdamW.

Evaluate both task improvement and important retained capabilities. A narrow dataset can improve the target metric while degrading broader behavior. If the model is used for generation, include representative outputs and failure cases, not only teacher-forced loss.

DECISION: State whether you are training from scratch, continuing pretraining, supervised fine-tuning or adapting a small parameter subset. These are different regimes, and hyperparameter advice should identify which one it addresses.

Full tuning versus frozen components

Full fine-tuning updates all trainable model weights. Freezing some layers reduces the number of updated tensors and optimizer states, but does not necessarily eliminate all forward or activation costs. The exact savings depend on where gradients are needed and how the graph is built.

A frozen-backbone linear probe can test whether existing representations already contain useful task information. Unfreezing additional layers can increase adaptation capacity, but it also raises memory and overfitting risk. These experiments answer different questions from changing the pretrained architecture itself.

Layerwise learning-rate decay assigns smaller rates to some layers, often earlier ones, while task-specific layers may receive larger rates. It can be useful, but it introduces another policy that needs validation. Begin with a simpler baseline unless the task or evidence motivates it.

CHECK: Verify requires_grad and optimizer groups. Reporting only trainable parameters is misleading if the full base model must still be stored and executed. Report both total and trainable counts.

LoRA rank is an adaptation-capacity choice

Low-rank adaptation represents a weight update as the product of two smaller matrices. For a base matrix W with output dimension d_out and input dimension d_in, an update BA with rank r uses r(d_in + d_out) parameters rather than d_out d_in, excluding any additional trained components.

The base weights are typically frozen. Rank limits the update's factorization, not the total rank or capability of the pretrained model. Target modules matter: adapting only query and value projections is different from adapting every attention and feed-forward projection.

Scaling conventions such as alpha/r affect update magnitude, and implementations can offer alternative scaling rules. Rank, alpha, learning rate and target modules interact. Compare a small number of ranks with a documented scaling convention instead of assuming that doubling rank alone always helps.

EXAMPLE: For a 512 by 512 matrix, rank 8 adds 8,192 adapter weights. Adapting four such matrices adds 32,768, before other components. This arithmetic does not include storage for the frozen base weights.

Fine-tuning data deserves special scrutiny

Small adaptation datasets can contain duplicates, inconsistent formatting or examples overlapping evaluation. With few examples, these issues can dominate the apparent benefit of a learning-rate or rank change. Split at the appropriate source or conversation level and inspect representative records.

Loss masking is especially important when only part of a sequence should be supervised. For example, an instruction-style objective may train on response tokens while providing prompt tokens as context. The exact policy depends on the task; record it explicitly. Masking changes the valid-token denominator and the objective.

Use a limited number of passes initially and monitor validation. A model may memorize the fine-tuning set quickly. More epochs are not automatically better, and a lower rate does not make indefinite repeated training harmless.

DECISION: Choose adaptation capacity and duration to match the amount and diversity of supervision. When results are poor, inspect examples and target formatting before increasing LoRA rank or unfreezing the entire model.

Worked adaptation decision

You have a pretrained model and a small labeled task dataset. Full fine-tuning exceeds optimizer-memory limits. A low-rank adapter fits, but rank 32 has nearly the same validation score as rank 8 and takes more memory. Both use comparable tuning effort and the same target modules.

A reasonable choice is rank 8, unless another important metric favors the larger rank. The result shows that extra adapter capacity did not help under this dataset and recipe. It does not prove rank 8 is universally sufficient or that target-module selection is unimportant.

ANSWER: Prefer the simplest configuration meeting the objective. Record total model size, trainable adapter count, scaling convention, targeted matrices, learning rate, passes and evaluation protocol. Without these details, the rank number alone is almost meaningless.

FOLLOW-UP: If both ranks underperform, investigate learning rate, data quality, masking and whether the targeted modules allow the needed adaptation. Increasing rank is only one possible next experiment.

Training and decoding are separate choices

Training learns a conditional probability distribution. Decoding chooses how to turn that distribution into an output sequence. Temperature, top-k, top-p, beam size and length penalties affect generated outputs without changing the learned weights.

For fair training comparisons, keep decoding settings fixed. Otherwise a model may look more creative simply because its sampling temperature is higher. For deployment, decoding deserves its own evaluation after a model is selected. The best settings depend on whether you want faithful translation, deterministic answers or diverse creative outputs.

Greedy decoding picks the highest-probability next token at every step. Sampling draws from a modified distribution. Beam search keeps several candidate continuations according to a scoring rule. None is universally best for every task or metric.

DECISION: Do not use decoding changes to claim a training improvement. A repetitive model can look less repetitive under stronger randomness while becoming less accurate or coherent. Evaluate the actual target behavior.

Temperature changes concentration

For logits z_i and positive temperature tau, sampling probabilities are proportional to exp(z_i/tau). A smaller temperature makes the distribution more concentrated; a larger one makes it flatter. Temperature approaching zero is associated with greedy-like selection, but implement greedy decoding explicitly rather than dividing by zero.

Top-k sampling restricts to a fixed number of highest-probability tokens. Top-p sampling keeps a smallest leading set whose cumulative probability reaches a threshold, with implementation details affecting the exact boundary. These methods change the support from which sampling occurs.

EXERCISE: If two logits differ by 2, their probability ratio before other filtering is exp(2/tau). At tau = 1 the ratio is about 7.39; at tau = 2 it is about 2.72. Higher temperature reduces the preference.

CHECK: Record the order of temperature scaling and filtering if the implementation permits alternatives. The same numeric top-p and temperature can behave differently under different processing pipelines.

Beam search has its own tradeoffs

Beam search can improve sequence search for some structured generation tasks, including translation, but increasing beam size is not guaranteed to improve human quality. Length normalization, end-token handling and model calibration affect the ranking of completed sequences.

Larger beams increase computation and memory, and can interact with length bias. Evaluate a few beam sizes with fixed model weights and a consistent scoring rule. Do not choose the beam solely by inspecting one attractive example.

For open-ended language generation, beam search can produce repetitive or generic text because high-probability sequences do not necessarily match the desired diversity. Sampling may better fit that use case. Task requirements determine the decoding objective.

DECISION: Tune decoding on a validation set appropriate to deployment, then freeze it for final evaluation. If you change both the model and beam policy, report the combined system improvement rather than attributing all gains to architecture.

KV cache makes architecture matter at inference

Autoregressive decoding can cache keys and values from previous positions so they need not be recomputed at every new token. A rough self-attention cache element count is 2 times B times L times T times h_kv times d_h. Multiply by bytes per element for an approximate storage cost.

For standard MHA, h_kv d_h = d, giving about 2BLTd elements. GQA reduces h_kv relative to query heads and can reduce this cache. Cross-attention and other serving allocations require additional accounting.

EXAMPLE: B = 1, L = 12, T = 1,024, d = 640 and two-byte cache elements give 31,457,280 bytes, about 30 MiB, for this simplified standard-attention self-cache. This excludes weights, activations, allocator overhead and other buffers.

DECISION: Include generation memory and latency when choosing width, depth and attention variant. A training configuration that fits comfortably may still be unsuitable for many simultaneous long-context requests.

Worked deployment choice

Two models have nearly equal validation loss. Model A is deeper and narrower; model B is shallower and wider. Their parameter counts are similar, but measured single-sequence generation latency favors B on your target device. A may still have different batch-throughput behavior.

If interactive latency is the primary deployment constraint, B is the stronger current choice, assuming task quality and memory are acceptable. If you serve large batches, benchmark that scenario separately. One hardware workload does not predict every other workload.

ANSWER: Choose architecture with the actual serving pattern in mind: prompt length, generated length, batch size, concurrency and cache dtype. Parameter count is an incomplete proxy for user experience.

FOLLOW-UP: Tune decoding after selecting a model, with fixed prompts and an appropriate quality protocol. Report both quality and speed. A faster system that produces worse answers may not meet the task even if its tokens-per-second number is impressive.

A scheduler with explicit update semantics

The function below returns the rate for a one-based optimizer update k. Updates 1 through W warm up linearly; update W reaches the peak. The final update S reaches the floor. With W = 0, update 1 starts at the peak. This is a defined convention, not an assertion about every framework scheduler.

```
import math

def lr_at(k, total, warm, peak, floor):
    assert total >= 2
    assert 0 <= warm < total
    assert 1 <= k <= total
    assert 0 <= floor <= peak
    if warm and k <= warm:
        return peak * k / warm
    start = warm if warm else 1
    progress = (k-start) / (total-start)
    return floor + 0.5*(peak-floor)*(
        1 + math.cos(math.pi*progress))
```

Set each parameter group's rate immediately before that update, or adapt the function to the framework's scheduler convention. Preserve relative multipliers if groups have different rates. Test the endpoints and plot the whole schedule before training.

CHECK: Do not call this once per microbatch when accumulation is intended. The function's k is an optimizer-update index, not a data-loader iteration index.

Token-weighted accumulation pattern

This PyTorch-style integration sketch assumes one device, ordinary unscaled backward, and losses returned as sums over valid targets. It is not a standalone training program. Adapt tensor names and ignore_index to your model. It deliberately omits mixed-precision scaling and distributed normalization, which require additional handling.

```
optimizer.zero_grad(set_to_none=True)
valid_total = 0
for batch in microbatches:
    logits = model(batch['x'])
    y = batch['y']
    loss_sum = F.cross_entropy(
        logits.reshape(-1, logits.size(-1)),
        y.reshape(-1), ignore_index=-100,
        reduction='sum')
    loss_sum.backward()
    valid_total += (y != -100).sum().item()
assert valid_total > 0
for p in model.parameters():
    if p.grad is not None:
        p.grad.div_(valid_total)
preclip = torch.nn.utils.clip_grad_norm_(
    model.parameters(), max_norm=1.0)
optimizer.step()
```

The clip value is illustrative. This scheme also handles a final partial accumulation group using its actual denominator. In distributed training, account for how gradient synchronization averages or sums across devices; do not apply this one-device normalization unchanged without checking.

Evaluate with a sum and a denominator

A reliable evaluation loop aggregates summed negative log-likelihood and valid-token count, then divides once. The sketch below assumes the same one-device tensor interface as the previous page and a model that was training when evaluation began.

```
model.eval()
total_nll = 0.0
total_tokens = 0
with torch.no_grad():
    for batch in validation_loader:
        logits = model(batch['x'])
        y = batch['y']
        nll = F.cross_entropy(
            logits.reshape(-1, logits.size(-1)),
            y.reshape(-1), ignore_index=-100,
            reduction='sum')
        total_nll += nll.item()
        total_tokens += (y != -100).sum().item()
assert total_tokens > 0
val_loss = total_nll / total_tokens
perplexity = math.exp(val_loss)
model.train()
```

For a reusable utility, preserve and restore the previous training/evaluation mode rather than always forcing training mode. In distributed evaluation, combine sums and counts correctly. Avoid double-counting examples introduced by sampler padding. Very large losses can overflow exponentiation; report the loss and handle that numerical case explicitly.

Save the state that defines continuation

A checkpoint for faithful continuation needs more than model weights. Include optimizer state, scheduler or schedule definition, completed optimizer updates, processed-token count, random states, precision-scaler state when used and data-sampler progress where feasible. Save the resolved configuration and code/data identifiers alongside it.

```
state = {
    'model': model.state_dict(),
    'optimizer': optimizer.state_dict(),
    'completed_updates': update_count,
    'tokens_seen': tokens_seen,
    'config': resolved_config,
    'torch_rng': torch.get_rng_state(),
    'python_rng': random.getstate(),
}
# Also save CUDA/NumPy RNG states if used,
# scheduler/scaler state and sampler progress.
torch.save(state, checkpoint_path)
```

This is a schema sketch, not a complete crash-safe checkpoint manager. Use a temporary file and atomic replacement where supported to avoid corrupting the last good checkpoint. Test restoration in your environment. Exact replay can be limited by nondeterministic kernels, worker prefetching and distributed execution; state those limits rather than promising bitwise identity.

Numerical checks you can run without a GPU

Use simple assertions to verify arithmetic before spending training compute. The following checks cover the worked model count, token batch and scheduler boundaries. They do not test a complete Transformer implementation.

```
N = 32000*640 + 12*12*640**2
assert N == 79462400
assert 2*512*8*1 == 8192
assert abs(lr_at(100, 1000, 100,
    3e-4, 3e-5) - 3e-4) < 1e-12
assert abs(lr_at(1000, 1000, 100,
    3e-4, 3e-5) - 3e-5) < 1e-12
assert abs(lr_at(1, 1000, 0,
    3e-4, 3e-5) - 3e-4) < 1e-12
weighted = (100*4.0 + 900*2.0)/1000
assert abs(weighted - 2.2) < 1e-12
```

Add implementation-level checks for causal masking, target shift, parameter uniqueness, finite gradients and save/load behavior. A useful causal test alters future input tokens and confirms earlier-position outputs remain unchanged in evaluation mode, within numerical tolerance. Keep these tests small and meaningful; they protect the interpretation of every later hyperparameter experiment.

Your six-week learning route

Week one: read the decision, data and anatomy chapters. Build parameter and token-budget calculators, inspect a real batch and pass a tiny-batch overfit test. Your deliverable is a trustworthy baseline configuration, not a high score.

Week two: measure throughput and memory, then bracket learning rate and compare schedule behavior. Save the plots and explain why the chosen rate has a stability margin. Week three: compare a small number of width, depth and head configurations using explicit budgets and modest optimizer retuning.

Week four: diagnose generalization and test one targeted regularization or data intervention. Week five: run controlled continuation experiments, validate checkpoint recovery and evaluate generation with fixed decoding settings. Week six: confirm finalists, freeze the selection rule and write a reproducible report with an untouched test evaluation.

This is a flexible study route, not a promise that six weeks produces universal expertise. If a correctness issue takes longer, fixing it is progress. Skipping it to follow the calendar would undermine everything afterward.

Capstone A: choose an architecture

You have a fixed tokenizer with V = 16,000 and a limited GPU-time budget. Propose three conventional tied-decoder candidates: one small baseline, one wider candidate and one deeper candidate. Calculate main parameter counts, head dimensions, feed-forward widths and expected qualitative cost changes.

Specify which comparison is equal-token and which is equal-time. Give each architecture a comparable small learning-rate search. Measure actual throughput and peak memory rather than guessing from parameters. State a stopping rule for non-finite runs and a checkpoint-selection rule for valid runs.

YOUR DELIVERABLE: A table of configurations, a budget ledger, training/validation curves, cost measurements and a one-paragraph decision. Include at least one rejected candidate and explain what evidence rejected it. Do not fabricate results if you have not run the experiments; separate predictions from measurements.

SUCCESS STANDARD: Another person should understand why you chose the final configuration, what alternatives you tested and what uncertainty remains. A polished model-size number without this evidence does not meet the capstone objective.

Capstone B: rescue a stalled training run

Begin from a real checkpoint whose validation trend has stalled. Audit the learning-rate history, data pipeline, loss normalization, gradient norms and actual optimizer updates. Decide whether the apparent plateau is larger than ordinary noise and whether the current schedule has already reached its floor.

Design two continuations from the same checkpoint: an unchanged control and one justified intervention. The intervention might be a lower rate, a new decay phase or a corrected data issue. If you fix a bug, recognize that the comparison now concerns corrected training rather than pure hyperparameter tuning.

Hold additional token budget and evaluation policy fixed where possible. Save optimizer state consistently. Report both curves and select using the predefined rule. Explain why another intervention was not your first choice.

SUCCESS STANDARD: You can distinguish a plausible story from evidence. If the intervention does not help, your report should still identify what the result rules out and what experiment is now most informative.

Oral-exam questions with short answers

Why not always add heads? At fixed width, more heads make each head narrower and do not usually add main projection parameters. Quality and efficiency must be measured.

Why not always add blocks? More depth costs sequential computation and can need different optimization. The current model may be undertrained rather than too small.

Why not multiply AdamW learning rate by batch ratio? Batch changes gradient noise and update count, but no universal linear scaling rule guarantees the best adaptive-optimizer rate.

Why can validation loss be below training loss? Training-mode dropout, augmentation, different objectives or different data difficulty can cause it; compare equivalent evaluation conditions.

Why retain optimizer moments when reducing rate? Keeping them isolates the rate intervention and preserves the learned gradient statistics. Resetting them is a separate choice.

Why not compare character and subword perplexity? Their prediction units differ. Use a carefully defined common evaluation unit or a fixed tokenizer.

Why is a failed trial useful? It can reject a hypothesis, bound a stable region or expose a constraint, provided the configuration and failure are recorded.

The professional standard is justified uncertainty

You are ready to tune independently when you can explain what each important hyperparameter changes, estimate its cost, design a controlled comparison and interpret ambiguous results without inventing certainty. You should be able to say both what the evidence supports and what it does not.

Before a major run, answer: What is the target metric? What is fixed? What changes? How much compute will this consume? What would make me stop? Which checkpoint will I select? How will I know whether a difference is useful? Can I resume safely? These questions turn random choices into a repeatable practice.

No guide can remove every future doubt. New data, model families and hardware will create new questions. The durable skill is knowing how to resolve those questions through equations, diagnostics and experiments. That is more valuable than memorizing a list of preferred numbers.

FINAL ASSIGNMENT: Write a one-page justification for your next project's configuration before training it. After the run, annotate every prediction as supported, contradicted or unresolved. Use that record to choose the next experiment.

Glossary

Ablation: a controlled removal or alteration of one component to study its contribution. Architecture: the computation structure and its dimensions. Baseline: the documented reference configuration against which alternatives are compared.

Capacity: the model's representational resources, not a guarantee of usable skill. Generalization: performance on examples outside the training observations under the chosen distribution. Overfitting: improving fit to training data without corresponding held-out benefit.

Hyperparameter: a setting selected outside ordinary weight optimization. Parameter: a learned tensor value. Parameterization: how trainable quantities are represented and scaled.

Pilot: a limited-budget trial used to screen or diagnose. Seed: an initialization for a random-number generator. Checkpoint: saved state used for selection, recovery or continuation.

Teacher forcing: conditioning training predictions on true prior target tokens. Autoregressive generation: predicting sequentially using previously generated tokens. KV cache: stored keys and values reused during decoding. Gradient accumulation: combining microbatch gradients before an optimizer update.

Compute-optimal: optimal under a specified compute objective. Pareto tradeoff: improving one objective may worsen another. Search budget: resources spent selecting a configuration.

Warm start: initialize from existing weights. Resume: continue with the intended state. Restart: deliberately begin a new optimization phase.

Reading list 1 / 2

Established sources; links and current APIs were not live-verified during preparation.

01. Vaswani et al. (2017). Attention Is All You Need.

Original Transformer architecture, scaled dot-product attention and encoder-decoder design. Read alongside chapters 3, 7 and 10.

arxiv.org/abs/1706.03762

02. Kingma and Ba (2014). Adam: A Method for Stochastic Optimization.

Moment estimates, bias correction and the adaptive update rule. Background for chapters 12 and 17.

arxiv.org/abs/1412.6980

03. Loshchilov and Hutter (2017). Decoupled Weight Decay Regularization.

Why decoupled weight decay differs from an L2 gradient penalty under adaptive optimization. See chapter 17.

arxiv.org/abs/1711.05101

04. Hoffmann et al. (2022). Training Compute-Optimal Large Language Models.

Evidence for balancing parameters and training tokens under a compute objective. Do not treat fitted ratios as universal constants. See chapter 24.

arxiv.org/abs/2203.15556

05. Kaplan et al. (2020). Scaling Laws for Neural Language Models.

Empirical scaling relationships and their assumptions. Useful context for budget reasoning, not a substitute for local measurements.

arxiv.org/abs/2001.08361

06. Bergstra and Bengio (2012). Random Search for Hyper-Parameter Optimization.

Motivation for random search and why large grids can waste trials. See chapter 22.

www.jmlr.org/papers/v13/bergstra12a.html

Reading list 2 / 2

Established sources; links and current APIs were not live-verified during preparation.

07. Xiong et al. (2020). On Layer Normalization in the Transformer Architecture.

Analysis of normalization placement and optimization behavior. Background for chapter 11.

arxiv.org/abs/2002.04745

08. Shazeer (2020). GLU Variants Improve Transformer.

Gated feed-forward alternatives and parameter-aware comparison. Read with chapter 8.

arxiv.org/abs/2002.05202

09. Ainslie et al. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.

Grouped-query attention and the quality/efficiency motivation. See chapters 7 and 26.

arxiv.org/abs/2305.13245

10. Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models.

Low-rank update factorization and adaptation choices. See chapter 25.

arxiv.org/abs/2106.09685

11. Dao et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.

Why implementation and memory traffic matter, even for mathematically equivalent attention. See chapters 4 and 19.

arxiv.org/abs/2205.14135

12. PyTorch official documentation.

Check your installed version for AdamW, learning-rate schedulers, automatic mixed precision, gradient clipping, checkpointing and serialization. Code sketches here need integration and version checks.

pytorch.org/docs/stable/index.html

The tuning desk reference

Architecture: define task and budget first. For a standard tied decoder with plain f = 4d blocks, estimate N approximately Vd + 12Ld^2, then count actual tensors. Choose width and depth together. Record head dimension alongside head count.

Learning rate: start from a coherent nearby recipe, test a small logarithmic range and choose using validation plus stability. Plot the applied schedule. Retune after major changes. A short pilot proposes candidates; it does not prove an optimum.

Switching: define update or token units. Save full state before an intervention. Keep optimizer moments for a simple rate change. Verify that the scheduler preserves the new policy. Compare continuation branches when affordable.

Batch and loss: count valid tokens across microbatches, accumulation and devices. Normalize gradients correctly. Unscale before clipping. Advance schedules according to the intended actual progress measure.

Diagnosis: both losses high means check correctness and optimization before capacity. Training improves while validation worsens means inspect generalization. NaNs mean locate the first failing computation. A plateau alone does not identify the right intervention.

Evidence: compare under stated budgets, record failed trials, repeat close finalists where useful and keep the test separate. Report cost, quality and uncertainty together. Additional complexity should earn its place.

YOUR NEXT STEP: Build one reliable baseline. Write the hypothesis, controls, budget and stopping rule for your first learning-rate comparison. Then run it and let the evidence choose the next experiment.